Ιόνιο Πανεπιστήμιο

Τμήμα Πληροφορικής

– Πτυχιακή Εργασία –

Βελτιστοποίηση Κατανεμημένης Εκτέλεσης Επερωτήσεων SQL σε Περιβάλλοντα Πολλαπλών Μηχανών

Βίκτωρ Γιαννακούρης - Σαλαλίδης
Επιβλέπων: – Δημήτριος Τσουμάκος –

23 Μαρτίου 2017

Επιβλέπων

Δημήτριος Τσουμάκος, *Αναπληρωτής Καθηγητής,*
Ιόνιο Πανεπιστήμιο

Τριμελής Επιτροπή

Τσουμάκος Δημήτριος, *Αναπληρωτής Καθηγητής,*
Ιόνιο Πανεπιστήμιο
Σιούτας Σπυρίδων, *Αναπληρωτής Καθηγητής,*
Ιόνιο Πανεπιστήμιο
Στεφανιδάκης Μιχαήλ, *Αναπληρωτής Καθηγητής,*
Ιόνιο Πανεπιστήμιο

Στους γονείς μου Θανάση & Μαρία και στον αδερφό μου Σωτήρη

Περίληψη

Ο τομέας του Big Data αποκτά όλο και περισσότερη προσοχή από την κοινότητα της επιστήμης των υπολογιστών τόσο σε ακαδημαϊκό όσο και σε βιομηχανικό επίπεδο. Λόγω της ευρείας και συνεχής χρήσης του διαδικτύου ο όγκος των δεδομένων αυξάνεται με εκθετικό ρυθμό.
Αυτό έχει οδηγήσει στην ανάγκη για ανάπτυξη πληθώρας διαφορετικών συστημάτων που να μπορούν να διαχειριστούν αποδοτικά τους διαφορετικούς τύπους δεδομένων. Επιπλέον, πολλά εξ'αυτών των συστημάτων στοχεύουν στην υποστήριξη της γλώσσας SQL, καθώς πρόκειται για μία ευρέως γνωστή γλώσσα που οι περισσότεροι προγραμματιστές και επιστήμονες δεδομένων γνωρίζουν.
Είναι συχνό φαινόμενο ένας προγραμματιστής να χρειαστεί να εκτελέσει ερωτήματα που αναφέρονται πίνακες σε διαφορετικές μηχανές. Έχουν αναπτυχθεί ορισμένα συστήματα τα οποία προσφέρουν λύση σε αυτό το πρόβλημα. Ωστόσο, τα περισσότερα από αυτά παρουσιάζουν μειονεκτήματα στην απόδοση καθώς δεν εκμεταλλεύονται επαρκώς τα χαρακτηριστικά των διαφόρων μηχανών που αποτελούν το σύστημα.
Σε αυτή την εργασία παρουσιάζεται το MuSQLE, ένα σύστημα για την βελτιστοποίηση της εκτέλεσης επερωτήσεων σε περιβάλλοντα πολλαπλών μηχανών. Το MuSQLE διαθέτει ένα καινοτόμο *Engine API* με το οποίο απλουστεύεται η εγκατάσταση νέων μηχανών στο σύστημα. Επιπλέον, η σύνδεση αυτού του API με τον βελτιστοποιητή πολλαπλών μηχανών του MuSQLE δίνει την δυνατότητα για βελτιστοποίηση του ερωτήματος λαμβάνοντας υπόψιν το μοντέλο κόστους που διαθέτει η εκάστοτε μηχανή.
Με την εκτέλεση των κατάλληλων πειραμάτων, δείχνουμε πως το MuSQLE μπορεί να πετύχει καλύτερη απόδοση από τα συστήματα με τα οποία συγκρίνεται έως και μία τάξη μεγέθους.

Πρόλογος και Ευχαριστίες

Η παρούσα πτυχιακή εργασία εκπονήθηκε κατά την διάρκεια της ερευνητικής μου εργασίας στο Εργαστήριο Υπολογιστικών Συστημάτων (CSLab) του τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του Εθνικού Μετσοβίου Πολυτεχνείου.

Αρχικά θα ήθελα να πω ένα μεγάλο ευχαριστώ στον επιβλέποντα καθηγητή μου Δημήτριο Τσουμάκο, ο οποίος μου έδωσε το κίνητρο και την ευκαιρία να ασχοληθώ με έναν τόσο ενδιαφέρον επιστημονικό κλάδο. Υπό την συνεχή επίβλεψή και καθοδήγησή του έμαθα και βελτίωσα σημαντικά τις δεξιότητές μου σε πολλαπλά θέματα όπως αυτό της επιστημονικής γραφής και μου άνοιξε νέους δρόμους σε ερευνητικά θέματα της επιστήμης των υπολογιστών. Επιπλέον, αποτέλεσε για εμένα μέντορα καθ'όλη την διάρκεια της προπτυχιακής και ερευνητικής μου σταδιοδρομίας σε όλες τις δύσκολες αποφάσεις που χρειάστηκε να πάρω. Χωρίς την συνεχή υποστήριξή του σε όλα τα ζητήματα, η πορεία μου δεν θα ήταν η ίδια.

Επιπλέον, θα ήθελα να ευχαριστήσω ιδιαίτερα τον Διδάκτορα Νικόλαο Παπαηλίου με τον οποίο συνεργάστηκα στενά κατά την διάρκεια της ανάπτυξης του θέματος της παρούσας εργασίας, καθώς ήταν πάντα διαθέσιμος να με βοηθήσει σε οποιαδήποτε τεχνική και θεωρητική δυσκολία. Διότι ως γνωστόν, "Ο Optimizer είναι μία δύσκολη υπόθεση".

Ακόμα, θα ήθελα να ευχαριστήσω τον καθηγητή μου από το Ιόνιο Πανεπιστήμιο Σπύρο Σιούτα, ο οποίος μου έδωσε τον έναυσμα, την υποστήριξη και καθοδήγησή του στα πρώτα βήματα της ερευνητικής μου πορείας τα χρόνια που ήμουν προπτυχιακός φοιτητής στην Κέρκυρα.

Επίσης, θα ήθελα να ευχαριστήσω τους συνεργάτες και φίλους μου από το Εργαστήριο Υπολογιστικών Συστημάτων Ιωάννη Γιαννακόπουλο, Γιάγκο Μυτιλήνη και Χρήστο Μαντά για τις συζητήσεις, την συνεργασία και όλα όσα περάσαμε μέχρι σήμερα. Δίπλα τους έμαθα πολλά

και η πορεία των τελευταίων δύο χρόνων έγινε ακόμη πιο συναρπαστική.

Φυσικά, δεν θα μπορούσα να παραλείψω τον κολλητό μου Γιώργο Σμυρναίο καθώς και όλους τους φίλους μου από την Αθήνα καθώς και τους συμφοιτητές μου από την Κέρκυρα. Σας ευχαριστώ όλους για όλα τα αξέχαστα χρόνια που περάσαμε μαζί και τις ατελείωτες φιλοσοφικές και μη συζητήσεις. Ιδιαίτερα, θα ήθελα να ευχαριστήσω τους συμφοιτητές και φίλους μου Θάνο Ρέστα και Κατερίνα Μπόμπορη για όλη τους την στήριξη τον καιρό που έλειπα από το νησί.

Κλείνοντας, θα ήθελα να ευχαριστήσω τους γονείς μου για όλη την αγάπη και την υποστήριξη που μου προσέφεραν πάντα και συνεχίζουν να μου προσφέρουν μέχρι και σήμερα. Τους ευχαριστώ που είναι πάντα δίπλα μου σε όλες τις καλές και δύσκολες στιγμές. Οι συζητήσεις και η στήριξή τους σε κάθε μου βήμα έπαιξαν καθοριστικό σε όλα όσα έχω καταφέρει.

Βίκτωρ Γιαννακούρης - Σαλαλίδης
23 Μαρτίου 2017

Περιεχόμενα

Α΄ Εισαγωγή **1**

Β΄ Προαπαιτούμενα **4**

Β΄.1 Spark 4

Β΄.1.1 Resilient Distributed Datasets 5

Β΄.1.2 Fault Tolerance 6

Β΄.1.3 Spark SQL 6

Γ΄ Συσχετιζόμενες Εργασίες **10**

Γ΄.1 BigDAWG 10

Γ΄.2 MISO 13

Γ΄.3 CloudMdsQL 15

Δ΄ Το σύστημα MuSQLE **17**

Δ΄.1 Αρχιτεκτονική 17

Δ΄.1.1 Catalog 18

Δ΄.1.2 SQL Query Parser 19

Δ΄.1.3 Engine API 20

Δ΄.2 Βελτιστοποιητής Ερωτήματος 23

Δ΄.2.1 Ο αλγόριθμος DPhyp 23

Δ΄.2.1.1 csg-cmp-pair 23

Δ΄.2.1.2 Βελτιστοποίηση βάσει τοποθεσίας 24

Δ΄.2.1.3 Η μέθοδος emitCsgCmp 24

Δ΄.3 SQL Code Generator 26

Δ΄.4 Επεξεργασία ερωτήματος στο MuSQLE 26
Δ΄.4.1 Query Parsing 26
Δ΄.4.2 Multi-Engine Optimization 28
Δ΄.4.3 Εκτέλεση 29
Δ΄.5 Βελτιστοποιήσεις στο Spark SQL 29

Ε΄ Μοντέλα κόστους για το Spark SQL 31
Ε΄.0.1 Αριθμός Επαναλήψεων Παράλληλων Εργασιών 32
Ε΄.0.2 Κόστος Shuffling 32
Ε΄.0.3 Κόστος Broadcast Hash Join 33
Ε΄.0.4 Κόστος Sort-Merge Join 33

ΣΤ΄ Πειράματα 35
ΣΤ΄.1 Μηχανές 35
ΣΤ΄.1.1 MemSQL 35
ΣΤ΄.1.2 PostgreSQL 36
ΣΤ΄.1.3 Hadoop Distributed File System (HDFS) 36
ΣΤ΄.2 Υποδομή 37
ΣΤ΄.3 Σενάρια 38
ΣΤ΄.4 Απόδοση του Planner 39
ΣΤ΄.5 Ακρίβεια εκτίμησης κόστους 41
ΣΤ΄.6 Απόδοση εκτέλεσης ερωτήματος 42

Ζ΄ Συμπεράσματα 48

Η΄ Δημοσιεύσεις 49

Βιβλιογραφία 49

Κατάλογος σχημάτων

Β΄.1 Παράδειγμα δημιουργίας ενός Dataset από αρχείο Parquet 7

Δ΄.2 Η αρχιτεκτονική του MuSQLE 18
Δ΄.3 Παράδειγμα πίνακα του καταλόγου για τον date_dim 19
Δ΄.4 Ερώτημα Q 27
Δ΄.5 Ο γράφος του ερωτήματος Q 27
Δ΄.6 Το πλάνο εκτέλεσης του ερωτήματος Q 28

ΣΤ΄.7 Η αρχιτεκτονική του HDFS 38
ΣΤ΄.8 Απόδοση του planner για 3 μηχανές 40
ΣΤ΄.9 Απόδοση του planner για 6 μηχανές 41
ΣΤ΄.10 Ακρίβεια μοντέλου κόστους του Spark SQL 42
ΣΤ΄.11 Ακρίβεια μοντέλου κόστους της MemSQL 43
ΣΤ΄.12 Ακρίβεια μοντέλου κόστους της PostgreSQL 43
ΣΤ΄.13 Όλοι οι πίνακες σε όλες τις μηχανές - 5GB 44
ΣΤ΄.14 Κάθε πίνακας σε μία μηχανή - 5GB 45
ΣΤ΄.15 Κάθε πίνακας σε μία μηχανή - 20GB 47
ΣΤ΄.16 Κάθε πίνακας σε μία μηχανή - 50GB 47

Κατάλογος πινάκων

Δ΄.1 Τοποθεσίες των πινάκων του ερωτήματος Q 27

Ε΄.1 Παράμετροι Μοντέλου Κόστους 31

ΣΤ΄.1 Τοποθεσίες των πινάκων για το δεύτερο σενάριο πειραμάτων 39

Κεφάλαιο Α΄

Εισαγωγή

Τα τελευταία χρόνια ο τομέας του Big Data αποκτά ιδιαίτερα την προσοχή της κοινότητας της επιστήμης των υπολογιστών τόσο σε ερευνητικό όσο και σε βιομηχανικό επίπεδο. Ο όγκος των δεδομένων στο διαδίκτυο αυξάνεται εκθετικά με την πλέον ευρεία χρήση του για πολλαπλούς σκοπούς.

Ένα παράδειγμα είναι τα μέσα κοινωνικής δικτύωσης όπως Facebook, Twitter, Instagram κ.α. . Με εκατομμύρια χρήστες στο καθένα από αυτά υπάρχει συνεχής παραγωγή δεδομένων πολλών διαφορετικών τύπων όπως κειμένου, video και φωτογραφίες. Επιπλέον, με την χρήση των κινητών τηλεφώνων ο ρυθμός μεταβολής του όγκου των δεδομένων αυξάνεται ακόμα περισσότερο, καθώς οι χρήστες πολλές φορές μέσα στην ημέρα δημοσιεύουν στιγμιότυπα της καθημερινότητάς τους.

Όλα τα παραπάνω έχουν οδηγήσει στην ανάγκη εύρεσης νέων μεθόδων για την αποδοτική διαχείριση των δεδομένων μεγάλης κλίμακας. Ως αποτέλεσμα οι εφαρμογές του Big Data αποτελούν μεγάλο κομμάτι των μοντέρνων data centers. Ένα χαρακτηριστικό αυτών των εφαρμογών είναι η ετερογένεια των δεδομένων, καθώς η μορφή και ο τύπος διαφέρουν από πηγή σε πηγή λόγω της διαφορετικής εφαρμογής από την οποία παράγονται. Αυτό έχει οδηγήσει την κατασκευή πληθώρας διαφορετικών εφαρμογών και συστημάτων βάσεων δεδομένων για την αποδοτική διαχείριση των δεδομένων. Συνεπώς, για διαφορετικούς τύπους δεδομένων υπάρχουν διαφορετικά συστήματα με συγκεκριμένα χαρακτηριστικά ώστε να μπορούν να ανταποκριθούν στις ιδιαιτερότητες του κάθε διαφορετικού τύπου δεδομένων καθώς και σκοπού της εκάστοτε εφαρμογής.

Στις περισσότερες επιχειρήσεις υπάρχει παραπάνω από έναν σύστημα διαχείρισης δεδομέ-

νων για διαφορετικά τμήματα της εφαρμογής. Για παράδειγμα, ένα σύστημα διαχείρισης παραγγελιών μπορεί να αποτελείται από:
1. Μία σχεσιακή βάση δεδομένων(π.χ. PostgreSQL) για την οργάνωση των πελατών και
2. Μία αποθήκη δεδομένων(π.χ. Hive) για την αποθήκευση και ιστορική ανάλυση δεδομένων των παραγγελιών.

Για παράδειγμα, το Instagram[2] χρησιμοποιεί την PostgreSQL[3] για την διαχείριση των χρηστών και των μεταδεδομένων των φωτογραφιών, το Redis[4] για το activity feed και άλλες υπηρεσίες και το Amazon S3 (storage)[1] για την αποθήκευση των φωτογραφιών.

Είναι συχνό φαινόμενο σε αναλυτικές εφαρμογές ο προγραμματιστής να χρειάζεται να τρέξει ερωτήματα που να αναφέρονται σε πίνακες και δεδομένα σε διαφορετικές μηχανές προκειμένου να τα συνδυάσει και να εξάγει τα επιθυμητά αποτελέσματα. Αυτό μπορεί να οδηγήσει σε δύο προβλήματα:

1. **Ενοποίηση(Integration)**: Είναι αναγκαία η ανάπτυξη συστημάτων που να μπορούν να υποστηρίξουν την εκτέλεση SQL ερωτημάτων συνδυάζοντας δεδομένα από διαφορετικές μηχανές.

2. **Βελτιστοποίηση(Optimization)**: Έπειτα από την ενοποίηση των διαφορετικών συστημάτων σημαντικό είναι το ζήτημα της αποδοτικής εκτέλεσης αυτών των ερωτημάτων, αξιοποιώντας την κάθε μηχανή ξεχωριστά χωρίς να αγνοούνται οι πιθανές βελτιστοποιήσεις που προσφέρει η κάθε μία από αυτές.

Σε αυτή την πτυχιακή αναπτύχθηκε το σύστημα MuSQLE[9]. Το MuSQLE είναι ένα σύστημα ανοιχτού κώδικα[1] που στοχεύει στην αποδοτική εκτέλεση ερωτημάτων που αναφέρονται σε πίνακες που βρίσκονται σε πολλαπλές μηχανές διαφορετικών χαρακτηριστικών. Βασισμένο στο Spark SQL το MuSQLE υποστηρίζει την εκτέλεση τέτοιων ερωτημάτων εκμεταλλευόμενο την λογική της ενοποίησης των διαφορετικών μηχανών του Spark SQL καθώς και την μηχανή εκτέλεσης στης κύρια μνήμη με την χρήση RDDs και DataFrames. Επιπλέον, το MuSQLE διαθέτει ένα περιβάλλον διεπαφής, το Engine API, για την επικοινωνία του με τις εξωτερικές μηχανές. Αυτό δίνει τις εξής δυνατότητες:

- Ο προγραμματιστής μπορεί εύκολα να προσθέσει μία νέα μηχανή στο σύστημα υλοποιώντας 4 βασικές μεθόδους αντί να υλοποιήσει τον κάθε τελεστή(joins, aggregates κ.α.) της κάθε μηχανής ξεχωριστά.

[1]https://github.com/gsvic/MuSQLE

- Σε συνδυασμό με τον *βελτιστοποιητή πολλαπλών μηχανών* το MuSQLE αξιοποιεί τις εξωτερικές μηχανές στέλνοντας υποερωτήματα του αρχικού ερωτήματος για τοπική εκτέλεση σε αυτές, λαμβάνοντας μόνο το αποτέλεσμα του κάθε υποερωτήματος.

Στην συνέχεια της εργασίας γίνεται παρουσίαση σχετικών προσεγγίσεων στο θέμα της εννοποίησης και βελτιστοποίησης σε περιβάλλοντα πολλαπλών μηχανών. Στο κεφάλαιο της πειραματικής αξιολόγησης θα δειχθεί πως το MuSQLE μπορεί να επιτευχθεί βελτίωση στην χρονική απόδοση έως και μία τάξη μεγέθους σε σύγκριση με τα συστήματα με τα οποία συγκρίνεται.

Κεφάλαιο Β΄

Προαπαιτούμενα

Σε αυτήν την ενότητα παρουσιάζονται τα βασικά προαπαιτούμενα για την κατανόηση της παρούσας πτυχιακής εργασίας. Συγκεκριμένα, θα αναλυθεί το Spark SQL, στο οποίο και βασίστηκε το σύστημα MuSQLE.

Spark

Το Apache Spark[15] είναι ένα σύστημα κατανεμημένης υπολογιστικής που κατασκευάστηκε στο AMPLab στο πανεπιστήμιο της Καλιφόρνια στο Berkeley. Η βασική του συνεισφορά, είναι πως προσφέρει ένα προγραμματιστικό περιβάλλον για παράλληλη - κατανεμημένη επεξεργασία δεδομένων μεγάλης κλίμακας καθώς και ανοχή σε σφάλματα. Το Spark προσφέρει δυνατότητα επεξεργασίας δεδομένων με πολύ υψηλή απόδοση κατά κύριο λόγο στην κύρια μνήμη. Συγκριτικά με το αντίστοιχο MapReduce μοντέλο του Hadoop, προσφέρει ταχύτητα επεξεργασίας με έως και 100 φορές μεγαλύτερη απόδοση. Αυτό οφείλεται κυρίως στο ότι η επεξεργασία των δεδομένων λαμβάνει χώρα στην κύρια μνήμη (RAM) με την χρήση των δομών RDD. Το Spark αποτελείται από τα εξής 4 sub-projects:

- Spark Core: Η βασική δομή του Spark που δίνει την δυνατότητα για παράλληλη, στην κύρια μνήμη επεξεργασία των δεδομένων με την χρήση των RDDs[14].
- MLlib: Βιβλιοθήκη αλγορίθμων μηχανικής μάθησης
- GraphX: Βιβλιοθήκη επεξεργασίας γράφων

- Spark Streaming: Βιβλιοθήκη επεξεργασίας ρευμάτων δεδομένων(data streams))
- Spark SQL: Πρόκειται για ένα προγραμματιστικό περιβάλλον που προσφέρει κατανεμημένη εκτέλεση επερωτήσεων SQL σε πλήθος διαφορετικών σχεσιακών και μη συστημάτων. Με το Spark SQL [5] θα ασχοληθούμε κατά κύριο λόγο σε αυτή την εργασία, καθώς το σύστημα που υλοποιήθηκε βασίζεται κατά κύριο λόγο σε αυτό.

Resilient Distributed Datasets

Τα Resilient Distributed Datasets ή αλλιώς RDDs[14] αποτελούν τον πυρήνα του Spark καθώς και την υψηλότερου επιπέδου αφαιρετική του δομή. Πρόκειται για μία κατανεμημένη συλλογή εγγραφών, όπως μία λίστα ή ένας πίνακας. Μια αντιστοιχία θα μπορούσε να γίνει με μία συλλογή (collection) της Java η οποία τρέχει σε μία εικονική μηχανή JVM ενώ στο RDD υπάρχει μία αντίστοιχη κατανεμημένη συλλογή σε πολλά JVMs παράλληλα. Ένα RDD ορίζεται από πέντε χαρακτηριστικά:

- Μία λίστα από partitions. Ο αριθμός των partitions αποτελεί και τα tasks τα οποία θα τρέξουν παράλληλα, καθώς σε κάθε partition αντιστοιχεί και ένα task που το υπολογίζει
- Μία συνάρτηση που υπολογίζει το αποτέλεσμα του κάθε partition
- Μία λίστα από εξαρτήσεις σε άλλα RDDs
- Έναν Partitioner (προαιρετικό) ο οποίος ορίζει την λογική του partitioning, για παράδειγμα, hash-partitioning.
- Μία λίστα από τις προτιμόμενες τοποθεσίες υπολογισμού του κάθε partition, για παράδειγμα η τοποθεσία του block ενός αρχείου HDFS

Το RDD προσφέρει ανοχή σε σφάλματα (resilient) λόγω του γενεαλογικού γράφου με τον οποίο έπειτα από κάποιο σφάλμα μπορεί να επανυπολογιστεί από κάποιο checkpoint, κατανομή των εγγραφών σε πολλούς κόμβους(distributed) καθώς και έναν τύπο δεδομένων (dataset), για παράδειγμα: RDD[String] ή RDD[Int]. Άλλα χαρακτηριστικά του RDD είναι η δυνατότητά τους για εισαγωγή σε *κρυφή μνήμη (caching)*, είναι *read-only*, ακολουθούν την *lazy* λογική, που σημαίνει πως δεν υπολογίζεται τίποτα από αυτό μέχρι να γίνει κάποια κλήση που θα πυροδοτήσει τον υπολογισμό.

Fault Tolerance

Όπως προαναφέρθηκε, το Spark διαθέτει μηχανισμό ανοχής σε σφάλματα ο οποίος βασίζεται στην συντήρηση ενός γενεαλογικού γράφου. Αυτό δίνει την δυνατότητα της γρήγορης επαναφοράς χαμένων partitions από κόμβους στους οποίους συνέβη κάποιο σφάλμα επανυπολογίζοντάς τα. Αυτό είναι πολύ πιο αποδοτικό συγκριτικά με τις βασισμένες σε checkpoint τεχνικές καθώς η συντήρηση ενός γενεαλογικού γράφου απαιτεί σημαντικά λιγότερο χώρο και χρόνο σε σχέση με ένα checkpoint στον δίσκο, καθώς στην περίπτωση του πρώτου απαιτείται μόνο ο επανυπολογισμός των χαμένων partitions του RDD. Στην περίπτωση σφάλματος στις checkpoint τεχνικές θα έπρεπε να επανυπολογιστεί ολόκληρο το RDD από την αρχή.

Spark SQL

Το Spark SQL είναι το κομμάτι του Spark Project που θα μας απασχολήσει περισσότερο σε αυτή την εργασία και δίνει την δυνατότητα της επεξεργασίας και επερώτησης δεδομένων με την χρήση της γνωστής SQL γλώσσας. Με το Spark SQL μπορούν να εκτελεστούν ερωτήματα σε δεδομένα που βρίσκονται είτε στην κύρια μνήμη σε RDDs είτε απο εξωτερικές πηγές δεδομένων όπως άλλες βάσεις δεδομένων (PostgreSQL, MemSQL, Cassandra κ.α.) ή κάποιο σύστημα αρχείων όπως το HDFS σε μορφές όπως JSON, Parquet και άλλες.

DataFrames

Το DataFrame αποτελεί την βασική αφαιρετική δομή του Spark SQL. Κάθε πίνακας εγγραφών, αντίστοιχος με έναν πίνακα σε μία σχεσιακή βάση, αναπαρίσταται με την χρήση ενός DataFrame. Επιπλέον, ένα DataFrame αναπαριστά και το πλάνο εκτέλεσης ενός ερωτήματος, στην απλούστερη περίπτωση, ένα απλό *scan* ή πιο συγκεκριμένα, ένα *LogicalRelation*.

Datasets

Ένα *Dataset* είναι μία πιο εξειδικευμένη μορφή του DataFrame. Ακολουθεί την αντίστοιχη λογική καθώς αναπαριστάται από ένα λογικό πλάνο του Catalyst και διαθέτει παρόμοιες μεθόδους. Η κύρια διαφορά είναι πως ένα Dataset αντιστοιχεί σε έναν συγκεκριμένο τύπο δεδομένων του JVM σε αντίθεση πε ένα DataFrame που αποτελεί μία συλλογή από εγγραφές τύπου *Row*. Επιπλέον, ένα Dataset διαθέτει έναν κωδικοποιητή ο οποίος σειριοποιεί τα αντικείμενα της συλλογής σε δυαδική μορφή. Και τα DataFrames καθώς και τα Datasets διαθέτουν μεθόδους για βασικές λειτουργίες αντίστοιχες με αυτές στις σχεσιακές βάσεις δεδομένων για

```
val customerDF = spark.read.parquet("customer.parquet")
val nationDF = spark.read.parquet("nation.parquet")
val regionDF = spark.read.parquet("region.parquet")

val finalDF = customerDF
    .join(nationDF)
    .where(customerDF("c_nationkey")
    .equalTo(n_nationkey))
    .join(region)
    .where(region("r_regionkey")
    .equalTo(nation("n_regionkey")))
```

Σχήμα Β΄.1: *Παράδειγμα δημιουργίας ενός Dataset από αρχείο Parquet*

πράξεις μεταξύ διαφορετικών σχέσεων όπως *joins*, *unions*, *selections* και *filters* καθώς και λειτουργίες τύπου *aggregation* όπως *count*, *sum*, *avg* και άλλες.

Query Planner

Όλα τα συστήματα βάσεων δεδομένων διαθέτουν έναν planner για τον σχεδιασμό του αποδοτικότερου πλάνου εκτέλεσης του δοθέντος ερωτήματος. Το Spark SQL διαθέτει τον *Catalyst* για αυτόν τον σκοπό. Συγκεκριμένα, ο Catalyst πραγματοποιεί βελτιστοποίηση του ερωτήματος βάσει κανόνων και συγκεκριμένων στρατηγικών από την κλάση *SparkStrategies*. Σε αυτή την κλάση βρίσκονται οι κανόνες που ακολουθούνται για ένα σύνολο πιθανών περιπτώσεων. Ένα παράδειγμα τέτοιου κανόνα είναι η επιλογή του αλγορίθμου συνένωσης: Στην περίπτωση που ο ένας από τους δύο πίνακες έχει μέγεθος σε bytes μικρότερο από ένα όριο (συγκεκριμένα, αυτό της παραμέτρου spark.sql.autoBroadcastJoinThreshold) τότε επιλέγεται ο αλγόριθμος *BroadcastHashJoin* ενώ σε αντίθετη περίπτωση ο *SortMergeJoin*. Ένα άλλο παράδειγμα είναι στην περίπτωση των φίλτρων, όπου εφαρμόζεται ένας κανόνας κατά τον οποίο ένα φίλτρο σε μία στήλη θα εφαρμοστεί στο χαμηλότερο δυνατό επίπεδο του δέντρου εκτέλεσης, δηλαδή πάνω από έναν κόμβο τύπου scan. Αυτός ο κανόνας είναι γνωστός στην βιβλιογραφία ως *filter pushdown*.

Αλγόριθμοι Join στο Spark SQL

Σε αυτή την ενότητα θα περιγραφούν ορισμένοι βασικοί αλγόριθμοι συνένωσης του Spark SQL. Οι αλγόριθμοι είναι εφαρμογές παραδοσιακών αλγορίθμων κατανεμημένων βάσεων δεδομένων. Για την μελέτη σε βάθος αυτών και πλήθος άλλων αλγορίθμων ο αναγνώστης παραπέμπεται στο βιβλίο [13]

Broadcast Hash Join

Αυτός ο αλγόριθμος επιλέγεται στην περίπτωση που το ένα από τα δύο DataFrames που πρόκειται να συνενωθούν είναι αρκετά μικρό ώστε να χωράει στην κύρια μνήμη. Συγκεκριμένα, αρκεί ο ένας από τους δύο πίνακες να έχει μέγεθος σε bytes μικρότερο της παραμέτρου *spark.sql.autoBroadcastJoinThreshold*. Τα βήματα του αλγορίθμου είναι τα εξής:

1. **Hash**: Σε αυτό το σημείο ο μικρότερος από τους δύο πίνακες κατακερματίζεται στο κλειδί με βάση το οποίο συνενώνεται με τον άλλο. Ο νέος κατακερματισμένος πίνακας ονομάζεται *HashedRelation*.

2. **Broadcast**: Στην συνέχεια, ο κατακερματισμένος πίνακα στέλνεται από τον master node σε όλους τους worker nodes με την λειτουργία broadcast του Spark. Το αποτέλεσμα αυτής της λειτουργίας έχει τον διαδεδομένο στους υπόλοιπους κόμβους πίνακα που ονομάζεται *broadcastRelation*.

3. **Join**: Στο τελευταίο βήμα εκτελείται το τελικό join το οποίο θα εκτελεστεί σε αριθμό διεργασιών που ισούται με τον αριθμό των partitions του μεγάλου πίνακα. Κάθε ένα από αυτά τα partitions θα συνενωθεί με τον μικρό με τη χρήση ενός Hash Join αλγορίθμου. Αφού ολοκληρωθεί αυτό έχουμε ως αποτέλεσμα τον τελικό συνενωμένο πίνακα.

Sort-Merge Join

Στις περιπτώσεις που και οι δύο πίνακες είναι αρκετά μεγάλοι επιλέγεται ο αλγόριθμος Sort-Merge Join. Σε αυτόν τον αλγόριθμο εφαρμόζεται σε κάθε πίνακα ένα sort-based shuffle και στην συνέχεια οι δύο πίνακες συνενώνονται κατά partitions. Τα βήματα είναι τα εξής:

1. **Shuffle**: Οι εγγραφές του πίνακα κατακερματίζονται ως προς το κλειδί της συνένωσης και ομαδοποιούνται βάσει αυτού σε range partitions με την εφαρμογή ενός αλγορίθμου hash partitioning.

2. **Sort**: Κάθε partition από τα παραπάνω ταξινομείται ως προς το κλειδί της συνένωσης με την χρήση external sorting.

3. **Merge**: Οι δύο πίνακες συνενώνονται κατά (ταξινομημένα πλεόν) partitions.

Πηγές Δεδομένων για το Spark SQL

Το Spark SQL δίνει την δυνατότητα σύνδεσης του συστήματός του με πληθώρα εξωτερικών μηχανών. Αυτό δίνει την δυνατότητα της διαλειτουργικότητας μεταξύ των μηχανών αλλά και της επεξεργασίας πινάκων που βρίσκονται σε αυτές. Αυτό επιτυγχάνεται καθώς το Spark SQL αναπαριστά οποιονδήποτε πίνακα σε εξωτερική μηχανή ως ένα DataFrame. Κατά συνέπεια, αφού οι πίνακες φορτωθούν στο σύστημα μπορεί να γίνει η οποιαδήποτε επεξεργασία μεταξύ αυτών με τη χρήση των μεθόδων που διαθέτει η κλάση DataFrame. Οι βασικές πηγές δεδομένων με τις οποίες μπορεί να συνδεθεί το Spark SQL είναι οι εξής:

- Αρχεία Parquet: Δυνατότητα φόρτωσης αρχείων τύπου Parquet είτε από το τοπικό σύστημα αρχείων είτε από το κατανεμημένο σύστημα αρχείων HDFS

- Αρχεία JSON: Δυνατότητα φόρτωσης αρχείων τύπου JSON είτε από το τοπικό σύστημα αρχείων είτε από το κατανεμημένο σύστημα αρχείων HDFS

- JDBC: Σύνδεση με συστήματα βάσεων δεδομένων που διαθέτουν οδηγό JDBC και φόρτωση των πινάκων ως DataFrames

- Hive Tables: Σύνδεση με το Hive με τη χρήση του Hive Metastore, δίνοντας την δυνατότητα της εκτέλεσης οποιουδήποτε ερωτήματος υποστηρίζει το Hive μέσω του Spark SQL

Κεφάλαιο Γ΄

Συσχετιζόμενες Εργασίες

Σε αυτό το κεφάλαιο παρουσιάζονται ορισμένες συσχετιζόμενες εργασίες που αντιμετωπίζουν το πρόβλημα της εκτέλεσης επερωτήσεων σε περιβάλλοντα πολλαπλών μηχανών. Συγκεκριμένα, θα αναλυθούν τρεις ερευνητικές εργασίες, δηλαδή τα BigDAWG, MISO και το σύστημα CloudMdsQL. Παρακάτω παρουσιάζονται οι γενικές περιγραφές καθώς και τα πλεονεκτήματα όσο και τα μειονεκτήματα του κάθε συστήματος καθώς και πως το σύστημα που προτείνεται σε αυτή την πτυχιακή μπορεί να καλύψει αυτά τα μειονεκτήματα.

BigDAWG

Το BigDAWG [7] είναι ένα σύστημα polystore που επικεντρώνεται στην εκτέλεση ερωτημάτων που αφορούν διαφορετικά μοντέλα δεδομένων καθώς και συστήματα βάσεων δεδομένων. Η λογική του συστήματος βασίζεται στα *islands of information* τα οποία θα αναλυθούν παρακάτω. Το κίνητρο του όλου συστήματος αφορά τρεις στόχους.

1. Ο πρώτος αφορά την **διαφάνεια της τοποθεσίας**. Αυτό δίνει την δυνατότητα στον χρήστη της εκτέλεσης επερωτήσεων που αφορούν δεδομένα σε περισσότερα από ένα διαφορετικά συστήματα χωρίς να χρειάζεται να έχει την γνώση του κάθε συστήματος ούτε να ασχοληθεί με το πως θα αναθέσει τους ξεχωριστούς φόρτους εργασίας σε κάθε σύστημα. Αυτό επιτυγχάνεται με την λογική των *islands of information* που το καθένα αποτελείται από ένα σύνολο μηχανών στα οποία μπορεί να εκτελεστεί ένα ερώτημα με μία κοινή SQL γλώσσα.

2. Ο δεύτερος σχετίζεται με την **σημασιολογική πληρότητα**. Αυτό σημαίνει πως ο χρήστης προσθέτοντας ένα νέο σύστημα στο BigDAWG δεν θα χάσει καμία από τις ξεχωριστές δυνατότητες που πιθανώς να προσφέρει το συγκεκριμένο σύστημα, πράγμα το οποίο επιτυγχάνεται με την προσέγγιση των πολλαπλών islands of information.

3. Ο τρίτος στόχος είναι η δυνατότητα του χρήστη να έχει πρόσβαση και να συνδυάζει δεδομένα που βρίσκονται σε πολλαπλά islands of information από μόνο ένα back-end. Για παράδειγμα, μία σύνθετη αναλυτική διαδικασία η οποία πραγματοποιεί έναν γρήγορο μετασχηματισμό Fourier (FFT) για καρδιολογικά δεδομένα κυματομορφών για κάθε ασθενή, δεδομένου πως τα δεδομένα των ασθενών βρίσκονται σε μία σχεσιακή βάση PostgreSQL ενώ τα καρδιολογικά δεδομένα σε μία SciDB.

Islands of Information

Πρόκειται για την βασική αφαιρετική δομή του BigDAWG με την οποία αλληλεπιδρά ο χρήστης με μία ή παραπάνω μηχανές διαχείρισης δεδομένων. Συγκεκριμένα, ένα island of information χαρακτηρίζεται από τα παρακάτω:

- Ένα **μοντέλο δεδομένων** το οποίο καθορίζει το σχήμα των δεδομένων. Για παράδειγμα ένα μοντέλο δεδομένων μπορεί να αφορά δεδομένα μορφής πινάκων ή σχεσιακά δεδομένα συνδεδεμένα μεταξύ τους μέσω κύριων και ξένων κλειδιών.

- Μία **γλώσσα επερωτήσεων**, για την ανάκτηση δεδομένων ενός συγκεκριμένου μοντέλου. Για παράδειγμα, η γλώσσα SQL για τις σχεσιακές βάσεις δεδομένων.

- Ένα **σύνολο συστημάτων διαχείρισης δεδομένων** για την εκτέλεση των ερωτήσεων που δίνονται σε ένα island. Επιπλέον, κάθε μηχανή αυτού του συνόλου διαθέτει και ένα *shim* για την αντιστοίχηση/μετάφραση της γλώσσας του island στην δική της γλώσσα.

Επεξεργασία Ερωτήματος

Ένα island λαμβάνει ένα ερώτημα σε μορφή κειμένου και αφού το επαληθεύσει το μετατρέπει σε ένα Abstract Query Tree (AST). Έπειτα διαιρεί αυτό το AST σε ένα σύνολο από υποερωτήματα. Κάθε υποερώτημα που αναφέρεται σε συγκεκριμένη μηχανή μετατρέπεται στην γλώσσα της μηχανής με την χρήση του shim της κάθε μηχανής και έπειτα εκτελείται τοπικά στην μηχανή. Ο φόρτος εργασίας του island σχετίζεται κατά κύριο λόγο με την ένωση των αποτελεσμάτων της κάθε μηχανής. Με τη χρήση αυτής της διαδικασίας το σύστημα εκμεταλ-

λεύεται τον optimizer και την μηχανή εκτέλεσης του κάθε συστήματος ξεχωριστά σε αντίθεση με το να αναλάμβανε όλη την εκτέλεση του ερωτήματος το BigDAWG εξολοκλήρου.

Επιπλέον, υπάρχουν περιπτώσεις στις οποίες ένας χρήστης χρειάζεται να εκτελέσει ερωτήματα που αφορούν πολλά islands. Το BigDAWG δίνει την δυνατότητα εκτέλεσης τέτοιων ερωτημάτων, δηλαδή **cross-island querying** με την χρήση των τελεστών SCOPE (π.χ. RELATIONAL) για την δήλωση του island που επιθυμεί να εκτελεστεί το ερώτημα και CAST για την μετατροπή του μοντέλου μίας σχέσης από ένα island σε ένα άλλο.

Βελτιστοποίηση Ερωτήματος

Το BigDAWG υποστηρίζει βελτιστοποίηση των ερωτημάτων είτε για περιβάλλοντα αποτελούμενα από ένα island είτε από πολλά, δηλαδή single-island planning καθώς και multi-island planning. Η βελτιστοποίηση των ερωτημάτων γίνεται με black-box τρόπο που σημαίνει πως ο τοπικός βελτιστοποιητής της κάθε μηχανής αγνοείται. Η συγκεκριμένη λογική ακολουθείται από το BigDAWG διότι σε πολλά συστήματα βάσεων δεδομένων δεν διατίθενται πληροφορίες σχετικές με τα στατιστικά των δεδομένων ή εκτιμήσεις του κόστους εκτέλεσης. Επιπλέον, η βελτιστοποίηση βάσει κόστους θα απαιτούσε την γνώση και τη συντήρηση ενός μοντέλου για κάθε πιθανό τελεστή της κάθε μηχανής του συστήματος.

Με την εφαρμογή όλων των παραπάνω χαρακτηριστικών το BigDAWG προσφέρει ένα εύχρηστο και επεκτάσιμο σύστημα για την εκτέλεση ερωτημάτων που αφορούν δεδομένα διαφορετικού τύπου και αποθηκευμένα σε διαφορετικές μηχανές. Επιπλέον, η λογική της βελτιστοποίησης των ερωτημάτων κάνει το σύστημα εύκολα συμβατό με μία καινούργια μηχανή που πρόκειται να προστεθεί ανεξαρτήτως χαρακτηριστικών και γνώσης αυτής από τον χρήστη. Ωστόσο, η λογική της black-box βελτιστοποίησης μπορεί σε ορισμένες περιπτώσεις να οδηγήσει σε μειωμένη απόδοση της εκτέλεσης του ερωτήματος. Ένας κύριος λόγος είναι πως υπάρχουν συστήματα τα οποία διατηρούν στατιστικά και μοντέλα κόστους τα οποία μπορεί να είναι αρκετά ακριβή, καθώς έχουν πλήρη γνώση του συστήματος στα οποία τρέχουν. Για παράδειγμα τέτοια δυνατότητα δίνουν συστήματα με την χρήση της μεθόδου explain, όπως η PostgreSQL. Με την black-box λογική τέτοια πιθανά χαρακτηριστικά αγνοούνται, ενώ θα μπορούσαν να συνεισφέρουν σε ένα ακόμη καλύτερο πλάνο εκτέλεσης. Αντίθετα, το MuSQLE που προτείνεται στην παρούσα εργασία με την χρήση της μεθόδου *getCost* επιστρέφεται το κόστος εκτέλεσης ενός ερωτήματος χρησιμοποιώντας τις λειτουργίες που προσφέρει η κάθε μηχανή. Με αυτόν τον τρόπο εκμεταλλεύεται τα πιθανά ακριβή στατιστικά και μοντέλα κόστους του κάθε συστήματος.

MISO

Μία άλλη προσέγγιση σχετική με τα polystores είναι το MISO(Multistore Online tuning)[11]. Πρόκειται για ένα σύστημα που οργανώνει τον φυσικό σχεδιασμό των συστημάτων ενός polystore, με την έννοια πως αποφασίζει που είναι προτιμότερο να βρίσκονται(σε ποιο σύστημα) τα δεδομένα και πραγματοποιεί τις ανάλογες μετακινήσεις. Σκοπός του συστήματος είναι να ελαχιστοποιήσει τις μετακινήσεις των δεδομένων, δηλαδή πινάκων ή ενδιάμεσων αποτελεσμάτων που πραγματοποιούνται κατά την εκτέλεση των ερωτημάτων. Το MISO διαθέτει έναν προσαρμοστικό μηχανισμό συνδυάζοντας τα παρακάτω χαρακτηριστικά:

- Δουλεύει *online*, με την έννοια πως ενώ ο χρήστης συνεχώς στέλνει και εκτελεί ερωτήματα το σύστημα αυτομάτως προσαρμόζεται στον φόρτο εργασίας βασισμένο στα ερωτήματα που λαμβάνει προκειμένου να κάνει τις ανάλογες μετακινήσεις.

- Ως βασικό στοιχείο του φυσικού σχεδιασμού χρησιμοποιεί τα **materialized views**. Αξιοποιώντας με σωστό τρόπο τα views το σύστημα μπορεί να έχει πολλά οφέλη. Ένα είναι πως τα materialized views ενθυλακώνουν το αποτέλεσμα του ερωτήματος αποθηκεύοντας το στον δίσκο. Έπειτα, λαμβάνοντας την σωστή απόφαση για το που θα αποθηκευτεί το view μπορεί να επιτευχθεί αποφυγή πολλών πιθανών μετακινήσεων δεδομένων κατά την εκτέλεση μελλοντικών ερωτημάτων.

- Βασίζεται μόνο στα ενδιάμεσα αποτελέσματα της επεξεργασίας των ερωτημάτων. Για παράδειγμα, το HDFS αποθηκεύει τα ενδιάμεσα αποτελέσματα προκειμένου να πετύχει ανοχή σε σφάλματα. Επιπλέον τα ενδιάμεσα αποτελέσματα των ερωτημάτων στα polystores μετακινούνται από την μία μηχανή στην άλλη. Το MISO και στις δύο αυτές περιπτώσεις χρησιμοποιεί αυτά τα ενδιάμεσα αποτελέσματα ως ”καιροσκοπικά” materialized views και στρατηγικά τα αποθηκεύει στην ανάλογη μηχανή έτσι ώστε να πετύχει βέλτιστο χρόνο εκτέλεσης ερωτημάτων που θα εκτελεστούν μεταγενέστερα. Με αυτή την μέθοδο το MISO βελτιστοποιεί τον φυσικό σχεδιασμό των μηχανών ενός polystore χωρίς να χρειαστεί να εκτελέσει επιπλέον κοστοβόρες διαδικασίες.

Αρχιτεκτονική

Η αρχιτεκτονική του MISO είναι βασισμένη σε αυτή των mutlistores. Αποτελείται από δύο κύρια συστήματα, ένα Big Data Store και ένα RDBMS. Το πρώτο είναι το Hive(HV) ενώ το RDBMS

είναι μία παράλληλη αποθήκη δεδομένων(DW). Το σύστημα αποτελείται από τα παρακάτω χαρακτηριστικά:

- Datasets: Τα datasets, τα οποία αποτελούνται κυρίως από αρχεία log
- Queries: Ένα σύνολο από ερωτήματα $q_0, q_1, ..., q_n$ σε γλώσσα HiveQL τα οποία δίνονται ως είσοδος στο σύστημα
- Execution Layer, το οποίο είναι υπεύθυνο για την προώθηση των διάφορων μερών του query στις διάφορες μηχανές του συστήματος.
- Multistore Query Optimizer: Βελτιστοποίηση ενός ερωτήματος για ένα multistore. Ουσιαστικά επιλέγεται το βέλτιστο *split point* από το οποίο το πλάνο εκτέλεσης θα μετακινηθεί μαζί με τα ενδιάμεσα αποτελέσματα στο RDBMS και η εκτέλεση θα συνεχιστεί εκεί. Επιπλέον λαμβάνονται αποφάσεις και για τις κατάλληλες μετακινήσεις δεδομένων που θα συνεισφέρουν στο βέλτιστο πλάνο εκτέλεσης.
- MISO Tuner: Αυτό το module καλείται περιοδικά και είναι υπεύθυνο για την αναδιοργάνωση όποτε χρειάζεται των materialized views, δηλαδή τον φυσικό σχεδιασμό. Η περιοδικότητα της φάσης της αναδιοργάνωσης μπορεί να βασίζεται στον χρόνο(ανά τακτά χρονικά διαστήματα), στα ερωτήματα(ανά n ερωτήματα) ή στην κατάσταση του συστήματος(π.χ. όταν αυτό είναι σε αδράνεια).

Το MISO προσδίδει μεγάλη βελτίωση στην απόδοση της εκτέλεσης των ερωτημάτων πραγματοποιώντας τις απαραίτητες μετακινήσεις των δεδομένων βελτιστοποιώντας τον φυσικό σχεδιασμό των μηχανών στις οποίες τρέχει. Το MISO πραγματοποιεί βελτιστοποίηση του ερωτήματος βάσει κόστους. Αυτό σημαίνει πως διατηρεί τις δικές του συναρτήσεις κόστους για την αξιολόγηση των αποφάσεων που θα πάρει. Για να το κάνει αυτό χρειάζεται να τρέξουν ορισμένα πειράματα προκειμένου να ρυθμιστούν οι παράμετροι των συναρτήσεων σωστά. Αντίθετα το MuSQLE με την σωστή υλοποίηση της μεθόδου *getCost* μπορεί να κάνει αυτές τις εκτιμήσεις κόστους χρησιμοποιώντας λειτουργικότητα της ίδιας της μηχανής χωρίς να χρειάζεται να διατηρεί δικές του συναρτήσεις κόστους. Επιπλέον το MISO επικεντρώνεται κατά κύριο λόγο σε δύο μηχανές, το Hive και μία αποθήκη δεδομένων. Η τρέχουσα υλοποίηση του MuSQLE αποτελείται από τρεις διαφορετικές μηχανές και με το Engine API δίνει την δυνατότητα για σύνδεση με οποιαδήποτε μηχανή που υποστηρίζει SQL γλώσσα.

CloudMdsQL

Μία άλλη προσέγγιση ακολουθείται από το CloudMdsQL[10]. Πρόκειται για έναν σύστημα το οποίο παρέχει σύνδεση και ενοποίηση μεταξύ διαφορετικών μηχανών σε ετερογενή περιβάλλοντα με μία κοινή SQL γλώσσα. Το συγκεκριμένο σύστημα υποστηρίζει τοπική εκτέλεση των υποερωτημάτων στις διαφορετικές μηχανές καλώντας για το καθένα από αυτά την SQL γλώσσα της μηχανής στην οποία πρόκειται να εκτελεστεί. Με αυτόν τον τρόπο μπορεί να εκμεταλλευτεί την πλήρη ισχύ της κάθε συνδεδεμένης μηχανής, κάτι το οποίο αποτελεί και την κύρια καινοτομία του.

Περιγραφή του συστήματος

Το CloudMdsQL είναι βασισμένο στην SQL γλώσσα, με την επέκταση πως ενσωματώνει στα υποερωτήματα που πρόκειται να εκτελεστούν σε μία εξωτερική μηχανή την μετάφρασή τους στην φυσική γλώσσα της συγκεκριμένης μηχανής. Το κοινό μοντέλο δεδομένων είναι βασισμένο στους παραδοσιακούς σχεσιακούς πίνακες με την επιπρόσθετη δυνατότητα για υποστήριξη και ενοποίηση διαφορετικών τύπων δεδομένων όπως για παράδειγμα JSON ή δεδομένα μορφής πίνακα. Η αρχιτεκτονική του είναι βασισμένη στο μοντέλο *mediator/wrapper* και παρουσιάζεται παρακάτω.

Αρχιτεκτονική

Η αρχιτεκτονική του συστήματος αποτελείται από δύο βασικά μέρη: **Master**: Ο οποίος έρχεται σε άμεση επαφή με την διεπαφή του χρήστη και αποτελείται από τα εξής χαρακτηριστικά:

- Query optimizer: Βελτιστοποίηση και σχεδιασμός του ερωτήματος
- Catalog: Διατηρεί πληροφορίες σχετικά με τους πίνακες και τα στατιστικά τους
- Communication processor: Επικοινωνία με τους workers

Worker: Ελέγχονται από τον master και εκτελούν τα βήματα της επεξεργασίας του ερωτήματος. Αποτελούνται από τα εξής χαρακτηριστικά:

- Communication processor: Επικοινωνία με τον master
- Query execution controller: Αρχικοποιεί και ελέγχει την εκτέλεση ενός ερωτήματος. Επιπλέον αλληλεπιδρά με το operator engine για την τοπική εκτέλεση ενός υποερωτήματος

σε μία μηχανή. Επιπλέον συγχρονίζει την εκτέλεση με αυτή από άλλους worker nodes όταν χρειάζεται ενδιάμεσα αποτελέσματα από αυτούς.

- Operator engine: Εκτελεί τους τελεστές του ερωτήματος σε πίνακες που έχει λάβει το σύστημα είτε από κάποιον wrapper είτε από το table storage.
- Wrapper: Αλληλεπιδρά με ένα σύστημα που είναι συνδεδεμένο μέσω της γλώσσας που διαθέτει, εκτελεί ένα υποερώτημα, το μετατρέπει σε μορφή πίνακα και το αποθηκεύει στο table storage.
- Table storage: Αποδοτική και ομοιόμορφη αποθήκευση δεδομένων σε μορφή πίνακα των ενδιάμεσων αποτελεσμάτων.

Το CloudMdSQL είναι μία καινοτόμα λύση για την βελτιστοποίηση ερωτημάτων σε multistores καθώς εκμεταλλεύεται την πλήρη ισχύ κάθε συνδεδεμένου συστήματος εκτελώντας τοπικά υποερωτήματα. Επιπλέον με την αρχιτεκτονική του mediator/wrapper δίνει την δυνατότητα για την ενοποίηση σχεδόν με οποιοδήποτε σύστημα, καθώς μεταφράζει ένα SQL υποερώτημα στην φυσική γλώσσα του κάθε συστήματος. Ωστόσο, κάτι τέτοιο απαιτεί αρκετό χρόνο καθώς η συγγραφή ενός wrapper είναι μία σύνθετη διαδικασία καθώς χρειάζεται να υλοποιηθεί η αντιστοιχία κάθε πιθανού τελεστή. Επιπλέον, για τον υπολογισμό του κόστους πρέπει ο ίδιος ο χρήστης να περιγράψει αναλυτικά όλες τις συναρτήσεις κόστους.

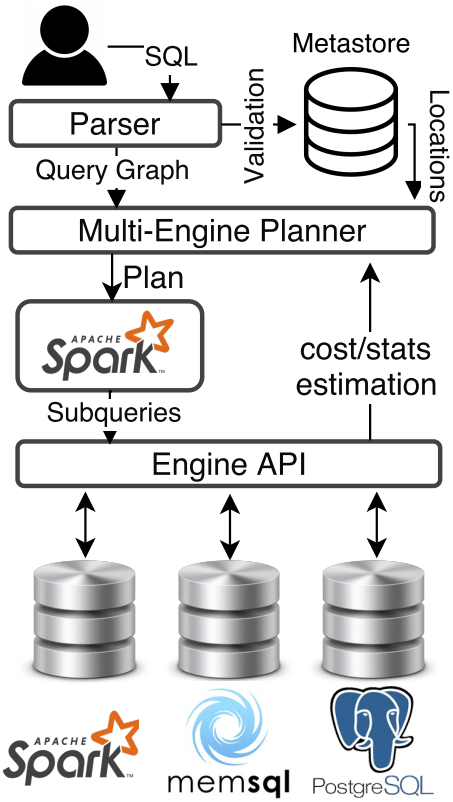
Κεφάλαιο Δ΄

Το σύστημα MuSQLE

Σε αυτό το κεφάλαιο περιγράφεται αναλυτικά το σύστημα MuSQLE που αναπτύχθηκε κατά την διάρκεια της εκπόνησης της παρούσας εργασίας. Το MuSQLE είναι ένα σύστημα που στοχεύει στην ταυτόχρονη βελτιστοποίηση της σειράς αλλά και της τοποθεσίας εκτέλεσης των υποερωτημάτων ενός ερωτήματος που αναφέρεται σε πίνακες που βρίσκονται σε περισσότερες από μία διαφορετικές μηχανές/συστήματα βάσεων δεδομένων. Κατά συνέπεια, βασικός σκοπός είναι η σωστή και ομοιογενής επικοινωνία μεταξύ του κεντρικού συστήματος και των διάφορων μηχανών-endpoints. Για αυτόν τον σκοπό διαθέτει ένα περιβάλλον διεπαφής το οποίο ονομάζεται Engine API το οποίο αναπτύχθηκε με σκοπό την αλληλεπίδραση του συστήματος με τις συνδεδεμένες σε αυτό μηχανές. Στην συνέχεια αναλύονται όλα τα επιμέρους τμήματα του συστήματος MuSQLE.

Αρχιτεκτονική

Το MuSQLE βασίστηκε στο τυπικό μοντέλο αρχιτεκτονικής μίας παραδοσιακής βάσης δεδομένων και διαθέτει πολλά κοινά στοιχεία όπως τον *Catalog*, *Query Parser* και τον *Planner/Optimizer*. Τα τρία προαναφερθέντα έχουν υλοποιηθεί με συγκεκριμένο τρόπο προκειμένου να μπορεί να υποστηριχθεί η βελτιστοποίηση και εκτέλεση ενός ερωτήματος σε περιβάλλον πολλαπλών μηχανών. Στην εικόνα Δ΄.2 παρουσιάζεται το σχήμα της αρχιτεκτονικής.



Σχήμα Δ΄.2: *Η αρχιτεκτονική του MuSQLE*

Catalog

Ο Catalog διατηρεί τα metadata όλων των πινάκων σε μία βάση δεδομένων και τους φορτώνει στο σύστημα κατά την εκκίνηση του. Συγκεκριμένα περιέχει πληροφορία σχετικά με τις διαφορετικές βάσεις στο σύστημα καθώς και τους πίνακες σε κάθε μία από αυτές με τα χαρακτηριστικά τους όπως τα ονόματα των στηλών και τους τύπους δεδομένων. Στο MuSQLE ο κατάλογος διατηρεί και πληροφορία σχετικά με την τοποθεσία του κάθε πίνακα. Το MuSQLE είναι δομημένο κατά κύριο λόγο με βάση το Spark. Συνεπώς, όλοι οι πίνακες σε όλες τις διαφορετικές μηχανές αντιμετωπίζονται ως DataFrames. Τα DataFrames μπορεί να αναφέρονται σε διαφορετικού τύπου *relation*, για παράδειγμα *JDBCRelation* στην περίπτωση που ο πίνακας βρίσκεται σε μία σχεσιακή βάση δεδομένων η οποία διαθέτει έναν οδηγό JDBC ή και *HdfsRelation* στην περίπτωση που ο πίνακας βρίσκεται σε κάποιο HDFS cluster. Στο MuSQLE κάθε πίνακας αναπαρίσταται με ένα αρχείο τύπου .mt (MuSQLE table) και αποτελείται από ένα JSON αρχείο που διατηρεί την εξής πληροφορία:

```
{
        tableName:"date_dim",
        tablePath:"date_dim",
        engine:"postgres",
        format:"parquet"
}
```

Σχήμα Δ΄.3: *Παράδειγμα πίνακα του καταλόγου για τον date_dim*

- tableName: Το όνομα του πίνακα
- tablePath: Το μονοπάτι του πίνακα (στην περίπτωση που βρίσκεται σε κάποιο σύστημα αρχείων όπως το HDFS. Στην περίπτωση μίας σχεσιακής βάσης αρκεί το όνομα του πίνακα)
- engine: Η μηχανή στην οποία βρίσκεται ο πίνακας (π.χ. postgres/spark)
- format: Στην περίπτωση που βρίσκεται σε κάποιο σύστημα αρχείων, ο τύπος δεδομένων, πχ parquet ή JSON. Στο σχήμα Δ΄.3 παρατίθεται ένα παράδειγμα ενός πίνακα του καταλόγου:

Κατά την εκκίνηση του συστήματος, το MuSQLE φορτώνει όλους τους πίνακες που βρίσκονται στον φάκελο *catalog* με την χρήση του *SparkSession*.

SQL Query Parser

Ως query parser χρησιμοποιείται αυτός που διαθέτει το Spark SQL. Ως είσοδος δίνεται ένα SQL ερώτημα σε μορφή κειμένου. Το Spark SQL χρησιμοποιεί τον Catalyst ως βελτιστοποιητή ερωτήματος. Κατά συνέπεια το Spark SQL επιστρέφει από αυτό το ερώτημα το αντίστοιχο δέντρο ερωτήματος σε μορφή *LogicalPlan*, μία δενδρική δομή δεδομένων του Catalyst. Έπειτα το LogicalPlan μετατρέπεται στον αντίστοιχο γράφο ερωτήματος προκειμένου να δοθεί ως είσοδος στον βελτιστοποιητή του MuSQLE.

Engine API

Σε αυτή την ενότητα παρουσιάζεται το περιβάλλον διεπαφής με τις μηχανές. Ουσιαστικά πρόκειται για το σημείο σύνδεσης του όλου συστήματος με τις εξωτερικές μηχανές. Μέσω αυτού το σύστημα έχει την δυνατότητα σε μία συγκεκριμένη μηχανή να εκτελέσει λειτουργίες όπως η εκτίμηση του κόστους εκτέλεσης ενός ερωτήματος, η εκτέλεση ενός υποερωτήματος, η μετακίνηση του αποτελέσματος ενός υποερωτήματος σε αυτή την μηχανή από μία άλλη κ.α. . Αναλυτικά, το Engine API ορίζεται από ένα *trait* της Scala με τις μεθόδους που αναλύονται παρακάτω.

Η μέθοδος getQueryCost

Αυτή η μέθοδος χρησιμοποιείται για την εκτίμηση του κόστους εκτέλεσης ενός ερωτήματος σε μία μηχανή. Η υλοποίηση της μεθόδου μπορεί να διαφέρει ανάλογα με την λειτουργικότητα που προσφέρει η εκάστοτε μηχανή. Για παράδειγμα, στην περίπτωση της Postgres, υπάρχει η μέθοδος *explain* η οποία επιστρέφει το πλάνο εκτέλεσης ενός ερωτήματος πριν αυτό εκτελεστεί. Επιπλέον, σε κάθε σημείο του πλάνου εκτέλεσης επιστρέφει το εκτιμώμενο κόστος εκτέλεσης με μονάδα μέτρησης τον αριθμό τον σελίδων από τον δίσκο που θα χρειαστεί να διαβαστούν στην κύρια μνήμη. Για την υλοποίηση της getQueryCost στην περίπτωση Postgres υλοποιήθηκε μία μέθοδος η οποία καλεί το explain clause και μετά από επεξεργασία της συμβολοσειράς που παράχθηκε απομονώνεται το κόστος εκτέλεσης. Έπειτα, με τεχνικές μηχανικής μάθησης μετατρέπεται αυτό το κόστος σε σελίδες σε χρονικό κόστος. Σε άλλες περιπτώσεις όπως αυτή του Spark SQL υλοποιήσαμε συναρτήσεις οι οποίες αναλύουν το κόστος ενός πλάνου και επιστρέφουν απευθείας το εκτιμώμενο χρονικό κόστος εκτέλεσης. Τα μοντέλα κόστους του Spark SQL αναλύονται περαιτέρω στο κεφάλαιο Ε΄. Παρακάτω δίνεται η περιγραφή της μεθόδου:

```
def getQueryCost(query: String): Double
```

Σύγκριση διαφορετικών μονάδων κόστους

Στις περισσότερες περιπτώσεις οι διαφορετικές βάσεις δεδομένων επιστρέφουν το κόστος εκτέλεσης του ερωτήματος σε διαφορετικές μονάδες μέτρησης. Για παράδειγμα το μοντέλο κόστους που αναπτύχθηκε στην εργασία για το Spark SQL επιστρέφει το κόστος σε χρόνο (δευτερόλεπτα). Η PostgreSQL επιστρέφει το κόστος εκτέλεσης σε disk page fetches. Για την

σωστή βελτιστοποίηση του ερωτήματος τα διαφορετικά κόστη από τις εξωτερικές μηχανές πρέπει να μεταφραστούν σε μία κοινή μονάδα μέτρησης προκειμένου να μπορούν να συγκριθούν μεταξύ τους. Σε αυτές τις περιπτώσεις χρησιμοποιούνται μοντέλα μηχανικής μάθησης. Συγκεκριμένα, για κάθε μηχανή εκτελείται ένα benchmark με μία σειρά από ερωτήματα όπου για το καθένα από αυτά κρατείται ο πραγματικός χρόνος εκτέλεσης σε δευτερόλεπτα καθώς και το εκτιμώμενο κόστος εκτέλεσης από την μέθοδο explain της κάθε μηχανής. Έπειτα, με την χρήση αυτών των δεδομένων γίνεται εκπαίδευση κάποιου στατιστικού μοντέλου όπου το X αναπαριστά το εκτιμώμενο κόστος ενώ το Y τον πραγματικό χρόνο σε δευτερόλεπτα, στην περίπτωση της PostgreSQL για παράδειγμα η *γραμμική παλινδρόμηση*. Κατά συνέπεια, για κάθε μηχανή υπάρχει το αντίστοιχο εκπαιδευμένο μοντέλο που μετατρέπει το εκτιμώμενο κόστος εκτέλεσης από οποιαδήποτε μονάδα μέτρησης σε δευτερόλεπτα.

Η μέθοδος getMoveCost

Η μέθοδος getMoveCost χρησιμοποιείται για την εκτίμηση του κόστους της μεταφοράς ενός πίνακα ή του αποτελέσματος ενός υποερωτήματος από μία μηχανή σε μία άλλη. Έπειτα από υπολογισμό του μεγέθους του αποτελέσματος με την χρήση στατιστικών το κόστος μεταφοράς μπορεί να υπολογισθεί με γραμμικό τρόπο από την συνάρτηση

$$C_{move} = B_t * T_{eng} \tag{Δ΄.1}$$

Όπου B_t το μέγεθος σε bytes αποτελέσματος t και T_{eng} ο ρυθμός μεταφοράς των bytes ανά δευτερόλεπτο στην μηχανή eng. Παρακάτω παρατίθεται η δήλωση της μεθόδου:

```
def getMoveCost(plan: DPJoinPlan): Double
```

Η μέθοδος injectStatistics

Όπως προαναφέρθηκε, ο planner του MuSQLE κατά τον σχεδιασμό του βέλτιστου πλάνου εκτέλεσης συμπεριλαμβάνει στα πιθανά πλάνα και αυτά που περιέχουν όλες τις δυνατές μετακινήσεις ενός πίνακα ή ενός ενδιάμεσου αποτελέσματος από μία μηχανή σε μία άλλη. Αυτό σημαίνει πως κατά την διαδικασία του planning θα υπάρξουν περιπτώσεις που θα κληθεί η συνάρτηση *getCost* σε μία μηχανή για υποερωτήματα που εμπεριέχουν πίνακες που δεν βρίσκονται σε αυτή την μηχανή αλλά υπάρχει περίπτωση να μετακινηθούν σε αυτή. Για παράδειγμα, έστω πως θέλουμε να υπολογίσουμε το κόστος εκτέλεσης του παρακάτω ερωτήματος στην

```
SELECT c_name, c_nation
FROM customer, nation, orders
WHERE c_nationkey = n_nationkey
AND c_custkey = o_orderkey
AND n_name = 'GERMANY'
```

PostgreSQL. Υποθέτουμε πως οι πίνακες customer και nation βρίσκονται στην PostgreSQL ενώ ο πίνακας orders στο HDFS. Στα στατιστικά της PostgreSQL υπάρχουν δύο χαρακτηριστικά που αναφέρονται στον αριθμό των γραμμών ενός πίνακα καθώς και στον αριθμό των σελίδων. Αυτά είναι τα *reltuples(αριθμός γραμμών)* και *pages* κατά αντιστοιχία. Προκειμένου να καλέσουμε την μέθοδο explain της PostgreSQL και να πάρουμε την εκτίμηση του κόστους εκτέλεσης του παραπάνω ερωτήματος, πριν την κλήση της *getCost* καλείται η συνάρτηση *injectStatistics* η οποία δημιουργεί έναν κενό πίνακα με το σχήμα του πίνακα orders και "αλλοιώνει" τα στατιστικά του στην PostgreSQL όπου στα μηδενικά *pages* και *reltuples* εισάγει τις πραγματικές τιμές. Η PostgreSQL δεν δίνει αυτή την δυνατότητα καθώς σε κάθε ερώτημα εισόδου ελέγχει τον πραγματικό αριθμό των pages στον δίσκο και κάνει βελτιστοποίηση βάσει αυτού του πραγματικού αριθμού. Προκειμένου λοιπόν να κάνουμε στατικά αυτά τα χαρακτηριστικά από τη στιγμή που τα θέτει το MuSQLE χρησιμοποιήσαμε το framework *pg_dbms_stats*[1] το οποίο δίνει την δυνατότητα για τροποποίηση και "πάγωμα" των στατιστικών. Η ίδια διαδικασία πραγματοποιείται και για όλα τα πιθανά ενδιάμεσα αποτελέσματα. Με αυτόν τον τρόπο μπορούμε να κάνουμε εκτιμήσεις τέτοιου τύπου ερωτημάτων στις διαφορετικές μηχανές εκτέλεσης του συστήματος. Παρακάτω παρατίθεται η δήλωση της μεθόδου injectStatistics:

```
def injectStatistics (plan: DPJoinPlan): Double
```

Η μέθοδος move

Η μέθοδος move δίνει την δυνατότητα στο σύστημα να πραγματοποιήσει μεταφορές ολόκληρων πινάκων ή και ενδιάμεσων αποτελεσμάτων από μία μηχανή σε μία άλλη. Υπάρχουν περιπτώσεις στις οποίες, για παράδειγμα, υπάρχει ένας πίνακας A στην PostgreSQL που πρέπει να συνενωθεί με τον πίνακα B στο Spark SQL. Στην περίπτωση αυτή, χρησιμοποιώντας τις μεθόδους getMoveCost καθώς και injectStatistics μπορούμε να υποθέσουμε που θα ήταν προ-

[1] https://github.com/ossc-db/pg_dbms_stats

τιμότερο να μετακινηθεί ένας από τους δύο πίνακες και στην συνέχεια να εκτελέσουμε όλο το ερώτημα στην μηχανή προορισμού. Αφού βρεθεί η βέλτιστη λύση εφαρμόζεται ένας τελεστής move στον πίνακα που πρόκειται να μετακινηθεί, με όρισμα την μηχανή προορισμού. Παρακάτω δίνουμε την δήλωση της μεθόδου move:

```
def move(plan: DPJoinPlan): Unit
```

Βελτιστοποιητής Ερωτήματος

Ο βελτιστοποιητής ερωτήματος είναι ένα από τα σημαντικότερα τμήματα ενός συστήματος βάσης δεδομένων, καθώς εξασφαλίζει την αποδοτικότερη εκτέλεση ενός ερωτήματος. Αυτό το πετυχαίνει βρίσκοντας την βέλτιστη σειρά με την οποία πρέπει να εκτελεστούν τα επιμέρους υποερωτήματα ενός ερωτήματος.

Ο αλγόριθμος DPhyp

Ως αλγόριθμος βελτιστοποίησης χρησιμοποιείται ο *DPhyp*[12], μία παραλλαγή του *DPccP*[8]. Πρόκειται για έναν αποδοτικό αλγόριθμο απαρίθμησης όλων των πιθανών πλάνων εκτέλεσης βασισμένο στις αρχές δυναμικού προγραμματισμού. Ο DPhyp μειώνει τον χώρο αναζήτησης πιθανών πλάνων εξετάζοντας όλους τους δυνατούς υπογράφους του αρχικού γράφου του ερωτήματος, οι οποίοι αποκαλούνται *csg-cmp-pairs*. Σε έναν τέτοιο γράφο κάθε κόμβος αναπαριστά έναν πίνακα ενώ κάθε ακμή αναπαριστά την συνένωση μεταξύ δύο πινάκων.

csg-cmp-pair

Έστω ο $G = (V, E)$ ένας γράφος συνένωσης και τα S1, S2 δύο υποσύνολα του V τέτοια ώστε το $S1 \subseteq V$ είναι ένας συνδεδεμένος υπογράφος και το $S2 \subseteq (V \setminus S1)$ ένα συνδεδεμένο συμπληρωματικό. Αν υπάρχει ακμή $(u, v) \in E$: $u \in$S1 και v$\in$S2, τότε αποκαλούμε το $(S1, S2)$ ένα *csg-cmp-pair*. Στην ουσία, τα csg-cmp-pairs είναι ζευγάρια αποτελούμενα από έναν συνδεδεμένο υπογράφο(csg) του ερωτήματος καθώς και ένα συνδεδεμένο συμπληρωματικό(cmp) του. Κάθε ζευγάρι αναπαριστά μία συνένωση μεταξύ των γράφων csg και cmp. Αυτή η ιδιότητα εξασφαλίζει πως ο αλγόριθμος θα εξετάσει όλους τους πιθανούς συνδυασμούς συνενώσεων.

Βελτιστοποίηση βάσει τοποθεσίας

Προκειμένου το σύστημα να είναι σε θέση να εκμεταλλευτεί τις δυνατότητες της κάθε μηχανής, ο βελτιστοποιητής χρειάζεται να λαμβάνει υπόψιν την τοποθεσία στην οποία βρίσκεται ο κάθε πίνακας. Για αυτόν τον λόγο τροποποιήθηκε η δομή του DPTable έτσι ώστε να μπορεί να χρησιμοποιηθεί και η πληροφορία της τοποθεσίας. Συγκεκριμένα, στην αρχική δομή του DPhyp το DPTable διατηρεί για κάθε πιθανό υπογράφο ένα πλάνο. Με την τροποποιημένη εκδοχή του DPTable για κάθε υπογράφο διατηρείται μία λίστα από πλάνα σε κάθε πιθανή μηχανή του συστήματος. Προκειμένου να γίνει απλή αυτή η διαδικασία, έχουν ενοποιηθεί ο βελτιστοποιητής με το Engine API. Αυτό σημαίνει πως σε κάθε βήμα της βελτιστοποίησης οι διάφορες εκτιμήσεις κόστους γίνονται καλώντας για κάθε μηχανή τις αντίστοιχες μεθόδους του Engine API. Με αυτόν τον τρόπο γίνεται απλή η διαδικασία την προσθήκης νέας μηχανής στο σύστημα υλοποιώντας μόνο τις μεθόδους του Engine API.

Η μέθοδος emitCsgCmp

Αυτή η συνάρτηση καλείται για κάθε πιθανό csg-cmp-pair και είναι υπεύθυνη για την εκτίμηση του κόστους της συνένωσης των S1 και S2 εφαρμόζοντας το βέλτιστο πλάνο του καθενός.
Έλεγχος μετακινήσεων: Όπως προαναφέρθηκε, το DPTable του αλγορίθμου είναι ένας πίνακας 2 διαστάσεων, όπου η δεύτερη διάσταση αναπαριστά τις πιθανές μηχανές στις οποία μπορεί να εκτελεστεί το πλάνο. Κατά συνέπεια για την δημιουργία του βέλτιστου πλάνου εκτέλεσης πρέπει να ληφθούν υπόψιν όλοι οι πιθανοί συνδυασμοί της συνένωσης των S1 και S2 (γραμμές 1-2) σε όλες τις διαθέσιμες μηχανές. Αυτό γίνεται στις γραμμές 1-26 όπου γίνονται οι έλεγχοι για τις εξής περιπτώσεις:

1. Μετακίνηση του $S1$ στην μηχανή του $S2$
2. Μετακίνηση του $S2$ στην μηχανή του $S1$
3. Μετακίνηση των $S1$ και $S2$ σε μία μηχανή e

Σε οποιαδήποτε από τις παραπάνω τρεις περιπτώσεις, προκειμένου να γίνει η εκτίμηση του κόστους ενός πλάνου S στην μηχανή e καλούνται οι μέθοδοι *getLoadCost* και *injectStats*, προκειμένου να υπολογιστεί το κόστος της μετακίνησης και να εισαχθούν τα στατιστικά του πλάνου S στην μηχανή e κατά αντιστοιχία. Στην συνέχεια, τα δύο νέα πλάνα $newPlan$ και $newPlan$ συνενώνονται στο τελικό πλάνο $newPlan$.

```
1  plans_1 = dpTable[S_1];
2  plans_2 = dpTable[S_2];
3  S = S_1 ∪ S_2;
4  p = ⋀_{(u_1,u_2)∈E, u_i⊂S_i} P(u_1, u_2) ;
5  for (e_1, plan_1) ∈ plans_1 do
6      for (e_2, plan_2) ∈ plans_2 do
7          for e ∈ engines do
8              //execute query in engine e
9              c_1 = 0; c_2 = 0;
10             newPlan_1 = plan_1; newPlan_2 = plan_2;
11             if e_1! = e then
12                 table_1 = newTempTable();
13                 c_1 = e.getLoadCost(plan_1.stats);
14                 e.injectStats(table_1, plan_1.stats);
15                 newPlan_1 = move(newPlan_1, e, table_1);
16             if e_2! = e then
17                 table_2 = newTempTable();
18                 c_2 = e.getLoadCost(plan_2.stats);
19                 e.injectStats(table_2, plan_2.stats);
20                 newPlan_2 = move(newPlan_2, e, table_2);
21             newPlan = newPlan_1 ⋈_p newPlan_2;
22             sql = toSQL(newPlan, e);
23             stats = e.getStats(sql);
24             stats.cost = stats.cost + c_1 + c_2;
25             if dpTable[S][e] = ∅ ∨ stats.cost < dpTable[S][e].cost then
26                 dpTable[S][e] = newPlan;
```

Algorithm 1: $emitCsgCmp(S1, S2)$

Μετατροπή σε κώδικα SQL: Έπειτα, το $newPlan$ μετατρέπεται σε κώδικα SQL με τη χρήση της μεθόδου *toSQL*(γραμμή 22). Για να γίνει αυτό, διασχίζεται όλο το δέντρο του πλάνου σταματώντας όπου συναντηθεί κάποιος τελεστής move, ο οποίος για το συγκεκριμένο πλάνο θεωρείται πίνακας ενδιάμεσου αποτελέσματος.

Έκτίμηση τελικού κόστους: Στο τέλος ο κώδικας που παράχθηκε δίνεται ως είσοδος στην μέθοδο *getStats*(γραμμή 24) και εκτιμάται το τελικό κόστος.

Ενημέρωση του DPTable: Στο τελευταίο βήμα, για το πλάνο που αντιστοιχεί στο σύνολο $S1 \cup S2$ γίνεται σύγκριση του κόστους της εκτέλεσης στην τρέχουσα μηχανή e' με την τελευταία μηχανή που ελέγχθηκε και βρίσκεται στο DPTable, e. Τελικώς, στο DPTable για το συγκεκριμένο πλάνο εισάγεται η μηχανή με το χαμηλότερο κόστος.

SQL Code Generator

Το MuSQLE επικοινωνεί με τις εξωτερικές μηχανές στέλνοντας σε αυτές ένα SQL ερώτημα και λαμβάνοντας μία απάντηση, για παράδειγμα, την εκτίμηση του κόστους. Κατά συνέπεια χρειάζεται ένας τρόπος να μετατρέπεται ένα *DPJoinPlan* σε SQL string προκειμένου να γίνουν οι αντίστοιχες κλήσεις του Engine API. Για αυτόν τον σκοπό υπάρχει ο SQL Code Generator. Αυτό το χαρακτηριστικό υλοποιείται στην κλάση *SQLCodeGen* και δίνει την δυνατότητα της μετατροπής ενός DPJoinPlan σε SQL string προκειμένου να γίνουν οι διάφορες κλήσεις στο Engine API για το κάθε πιθανό υποπλάνο. Για μία τέτοια μετατροπή αρκεί η κλήση της μεθόδου *genSQL(plan: DPJoinPlan)*.

Επεξεργασία ερωτήματος στο MuSQLE

Σε αυτή την ενότητα παρουσιάζονται τα βήματα της διαδικασίας επεξεργασίας ενός ερωτήματος στο MuSQLE. Περιληπτικά, ένα ερώτημα στο σύστημα διαιρείται σε όλα τα δυνατά υποερωτήματα καθώς και τις δυνατές μετακινήσεις. Έπειτα, το σύστημα στέλνει κάθε υποερώτημα στην βέλτιστη μηχανή για τοπική εκτέλεση. Έπειτα, το τελικό ερώτημα συνενώνει τα ενδιάμεσα αποτελέσματα, αν υπάρχουν, και εφαρμόζει και εκτελεί όλα τα aggregations του ερωτήματος στο Spark SQL, όπως για παράδειγμα *GROUP BY*, *ORDER BY* ή *SUM*. Παρακάτω παρουσιάζεται αναλυτικά η διαδικασία.

Query Parsing

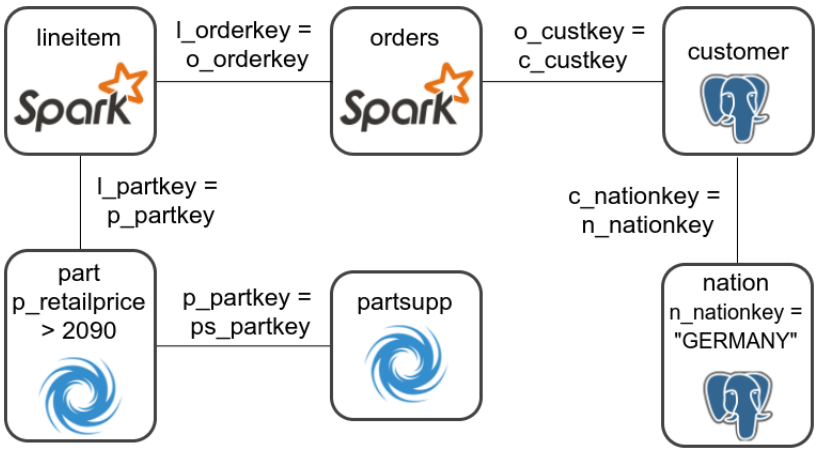
Υποθέτουμε πως θέλουμε να εκτελεστεί το ερώτημα Q που απεικονίζεται στην εικόνα Δ΄.4 για το οποίο οι τοποθεσίες των πινάκων φαίνονται στον πίνακα Δ΄.1. Αυτό το ερώτημα έπειτα από το parsing και το validation με την χρήστη του parser και του catalog θα έχει ως αποτέλεσμα τον γράφο του ερωτήματος στην εικόνα Δ΄.5. Σε αυτόν τον γράφο κάθε κόμβος αναπαριστά έναν πίνακα καθώς και την τοποθεσία του ενώ κάθε ακμή αναπαριστά μία συνένωση μεταξύ δύο πινάκων μαζί με την συνθήκη βάσει της οποίας γίνεται η συνένωση. Στην συγκεκριμένη περίπτωση όλες οι ακμές αναπαριστούν inner joins.

Table	Location
customer	PostgreSQL
nation	PostgreSQL
part	MemSQL
partsupp	MemSQL
orders	HDFS
lineitem	HDFS

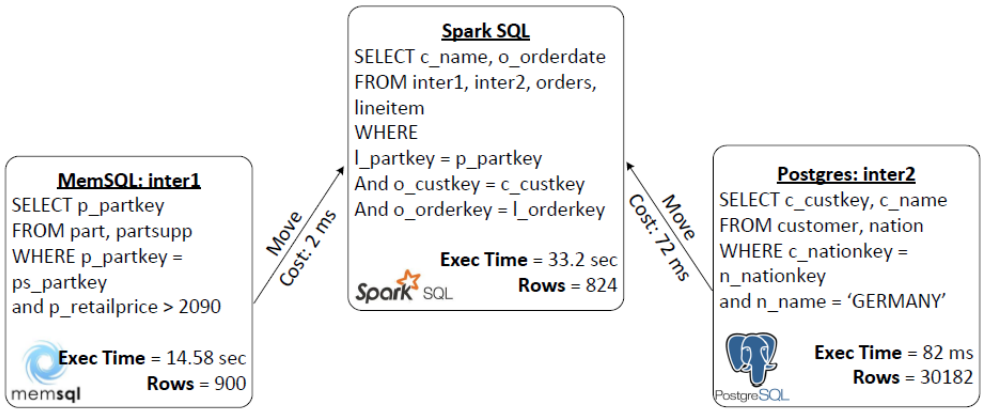
Πίνακας Δ΄.1: *Τοποθεσίες των πινάκων του ερωτήματος Q*

```
SELECT *
FROM customer, nation, orders, lineitem, part, partsupp
WHERE c_nationkey = n_nationkey
AND c_orderkey = o_orderkey
AND o_orderkey = l_orderkey
AND l_partkey = p_partkey
AND p_partkey = ps_partkey
AND p_retailprice > 2090
AND n_name = 'GERMANY'
```

Σχήμα Δ΄.4: *Ερώτημα Q*



Σχήμα Δ΄.5: *Ο γράφος του ερωτήματος Q*



Σχήμα Δ΄.6: *Το πλάνο εκτέλεσης του ερωτήματος Q*

Multi-Engine Optimization

Στην πορεία ο παραπάνω γράφος δίνεται ως είσοδος στον planner του συστήματος και ξεκινά η διαδικασία του optimization. Με εξαντλητικό τρόπο χρησιμοποιώντας δυναμικό προγραμματισμό με τον αλγόριθμο DPhyp, απαριθμούνται όλα τα πιθανά πλάνα εκτέλεσης και σε όλα τα βήματα γίνονται κλήσεις των μεθόδων του Engine API για τις διάφορες μηχανές όπως *getCost*, *injectStatistics* και *getMoveCost*. Στο τέλος επιστρέφεται το βέλτιστο πλάνο εκτέλεσης που αναπαρίσταται στην εικόνα Δ΄.6. Σε αυτό το πλάνο μπορούμε να δούμε πως το αρχικό ερώτημα έχει "σπάσει" σε τρία υποερωτήματα:

- Το ένα υποερώτημα συνενώνει τους πίνακες customer και nation καθώς και εφαρμόζει το φίλτρο στην στήλη n_name. Έπειτα στέλνεται για τοπική εκτέλεση στην PostgreSQL.
- Ένα άλλο υποερώτημα συνενώνει τους πίνακες part και partsupp και εφαρμόζει το φίλτρο στην στήλη p_retailprice και στέλνεται για τοπική εκτέλεση στο cluster της MemSQL.
- Έπειτα από την εκτέλεση των προηγούμενων δύο υποερωτημάτων τα δύο αποτελέσματα επιστρέφονται στο Spark SQL ως DataFrames με ονόματα inter1 και inter2. Στην πορεία το τελευταίο υποερώτημα εκτελείται στο Spark SQL το οποίο συνενώνει τα ενδιάμεσα αποτελέσματα inter1 και inter2 μαζί με τους δύο μεγαλύτερους πίνακες που βρίσκονται στο HDFS orders και lineitem. Το τελικό αποτέλεσμα επίσης επιστρέφεται ως DataFrame.

Εκτέλεση

Όπως προαναφέρθηκε το τελικό πλάνο εκτέλεσης επιστρέφεται ως ένα νέο DataFrame. Κατά συνέπεια για την εκτέλεση του αρκεί η κλήση μίας μεθόδου που θα πυροδοτήσει την εκτέλεση του DataFrame όπως η *take()* ή *collect()* ή κάποια μέθοδος aggregation όπως η *count*.

Βελτιστοποιήσεις στο Spark SQL

Προκειμένου να επιτευχθεί μεγαλύτερη απόδοση στην εκτέλεση των ερωτημάτων υλοποιήσαμε τις εξής βελτιστοποιήσεις στο Spark SQL που αναλύονται παρακάτω.

Επιλογή του βέλτιστου αλγορίθμου συνένωσης

Ένα σημαντικό μειονέκτημα του Spark SQL είναι πως όταν εμπλέκονται πίνακες που βρίσκονται σε κάποια σχεσιακή βάση ως *JDBC Relations* το Spark SQL δεν λαμβάνει υπόψιν το πραγματικό μέγεθος του πίνακα. Κατά συνέπεια όταν ο πίνακας εμπλέκεται σε κάποιο join ο αλγόριθμος που επιλέγεται αυτομάτως είναι ο *Sort-Merge Join*. Αυτό σε ορισμένες περιπτώσεις μπορεί να οδηγήσει σε μειωμένη απόδοση καθώς χρειάζεται να ταξινομηθούν και οι δύο πίνακες πριν γίνει το join. Σε περιπτώσεις που ο ένας πίνακας είναι αρκετά μικρός μπορεί να εφαρμοστεί ο αλγόριθμος *Broadcast-Hash Join* ο οποίος θα μπορούσε να πετύχει πολύ μεγαλύτερη απόδοση σε μία τέτοια περίπτωση.
Για να λυθεί αυτό το πρόβλημα γράφτηκαν κάποιες μέθοδοι για το MuSQLE που με την χρήση *reflection*[2] μπορεί κατά την διάρκεια της εκτέλεσης να εισαχθεί στο object του JDBCRelation το πραγματικό μέγεθος του πίνακα από τον Catalog. Με αυτόν τον τρόπο στις περιπτώσεις που ο ένας JDBC πίνακας είναι αρκετά μικρός, συγκεκριμένα, μικρότερος από το μέγεθος της παραμέτρου *spark.sql.autoBroadcastJoinThreshold*, εξαναγκάζουμε το Spark SQL να επιλέξει τον αλγόριθμο Broadcast-Hash Join, πετυχαίνοντας κατά αυτόν τον τρόπο σημαντικά μεγαλύτερη απόδοση.

Δυναμικό partitioning για σχέσεις τύπου JDBC

Στις περιπτώσεις που χρειάζεται να διαβαστούν πίνακες από μία εξωτερική βάση δεδομένων με την χρήση ενός JDBC driver το Spark SQL αυτόματα θα εκτελέσει αυτή την διαδικασία αναθέτοντάς την μόνο σε ένα partition, δηλαδή, όλος ο πίνακας θα διαβαστεί από μία διερ-

[2]https://docs.oracle.com/javase/tutorial/reflect/

γασία. Στις περιπτώσεις που ο πίνακας είναι αρκετά μεγάλος αυτό μπορεί να οδηγήσει στην αργή ανάγνωση του πίνακα καθώς στην πραγματικότητα δεν εφαρμόζεται παραλληλισμός. Επιπλέον, πολλοί πόροι του cluster μένουν ανεκμετάλλευτοι. Για την αντιμετώπιση αυτού του ζητήματος, στις περιπτώσεις αυτές και με την χρήση της μεθόδου *getStats* το MuSQLE λαμβάνει το μέγεθος του πίνακα. Έπειτα, θα ορίσει τον κατάλληλο αριθμό partitions έτσι ώστε να χωρίσει τον πίνακα σε chunks κατάλληλου μεγέθους ώστε το καθένα από αυτά να χωράει στην κύρια μνήμη, προκειμένου να αποφευχθεί πιθανό *disk spill*. Συγκεκριμένα, για έναν πίνακα μεγέθους S_{total} σε ένα cluster με M_{node} διαθέσιμη μνήμη και C_{node} επεξεργαστές ανά κόμβο, κάθε partition που τρέχει σε έναν πυρήνα καταλαμβάνει μνήμη ίση με

$$M_{CPU} = \frac{M_{node}}{C_{node}} \tag{Δ΄.2}$$

Κατά συνέπεια, το στάδιο της ανάγνωσης πρέπει να χωριστεί σε αριθμό partitions τέτοιο ώστε κάθε chunk πρέπει να έχει μέγεθος το πολύ έως M_CPU ώστε να χωράει στη μνήμη και να αποφευχθούν εγγραφές στον δίσκο. Συνεπώς, αυτός ο αριθμός partitions ορίζεται από την παρακάτω εξίσωση:

$$P_{JDBC} = \left\lceil \frac{S_{total}}{M_{CPU}} \right\rceil \tag{Δ΄.3}$$

Κεφάλαιο Ε΄

Μοντέλα κόστους για το Spark SQL

Προκειμένου να προστεθεί μία νέα μηχανή στο MuSQLE πρέπει να υλοποιηθούν οι μέθοδοι που αποτελούν το *Engine API*. Η μέθοδος *getQueryCost* χρησιμοποιείται για την εκτίμηση του κόστους εκτέλεσης ενός SQL ερωτήματος. Ενώ η MemSQL και η PostgreSQL διαθέτουν την συγκεκριμένη λειτουργία με την μέθοδο *explain* το Spark SQL δεν επιστρέφει το κόστος εκτέλεσης.

Για να προσθέσουμε το Spark SQL στο MuSQLE αναπτύξαμε ορισμένα μοντέλα κόστους για τους βασικούς αλγορίθμους join, για την φόρτωση δεδομένων από εξωτερικές πηγές καθώς και για την διαδικασία του shuffling(exchange).

Symbol	Description
D_r	Το κόστος ανάγνωσης μίας γραμμής
D_w	Το κόστος εγγραφής μίας γραμμής στον δίσκο
t_h	Το κόστος κατακερματισμού μίας τιμής
t_{br}	Το κόστος μετάδοσης μίας γραμμής
C_{cpu}	Το κόστος μίας σύγκρισης CPU
$cores$	Ο αριθμός πυρήνων στο cluster
$Part(s)$	Ο αριθμός των partitions της σχέσης s
$R(s)$	Ο αριθμός των γραμμών της σχέσης s
S_p	Η τιμή της παραμέτρου spark.sql.shuffle.partitions

Πίνακας Ε΄.1: *Παράμετροι Μοντέλου Κόστους*

Αριθμός Επαναλήψεων Παράλληλων Εργασιών

Στο Spark SQL κάθε εφαρμογή αποτελείται από έναν αριθμό σταδίων(stage) και το κάθε στάδιο από έναν αριθμό διεργασιών(tasks). Η πρώτη εξίσωση που θα αναλύσουμε είναι αυτή που περιγράφει τον αριθμό επαναλήψεων των παράλληλων διεργασιών που θα τρέξουν προκειμένου να ολοκληρωθεί ένα στάδιο. Κάθε εργασία μπορεί να δεσμεύσει έναν αριθμό CPUs που είναι δηλωμένος στην παράμετρο spark.task.cpus. Στην συγκεκριμένη υλοποίηση θέτουμε την παράμετρο σε 1. Ο αριθμός των διεργασιών που μπορούν να τρέξουν παράλληλα ισούται με τον αριθμό των επεξεργαστών που διαθέτει το cluster, ενώ ο αριθμός των διεργασιών που θα αποτελέσουν ένα στάδιο ορίζεται από τον αριθμό των *partitions*. Κατά συνέπεια, ο αριθμός των επαναλήψεων για την ολοκλήρωση ενός σταδίου αποτελούμενου από p partitions ορίζεται από την παρακάτω εξίσωση:

$$Rounds(p) = \left\lceil \frac{p}{cores} \right\rceil \tag{E΄.1}$$

όπου $cores$ ο αριθμός των διαθέσιμων επεξεργαστών στο cluster. Στην πορεία, το κόστος ενός σταδίου θα ορίζεται από το άθροισμα των μεγίστων χρόνων εκτέλεσης μίας διεργασίας t σε κάθε επανάληψη n, δηλαδή

$$\sum_{n=1}^{Rounds(p)} \max\left(C_t\right) \tag{E΄.2}$$

όπου C_t, το κόστος της διεργασίας t.

Κόστος Shuffling

Σε αυτή την διαδικασία όλες οι γραμμές κατακερματίζονται σε ένα κλειδί και στην πορεία ομαδοποιούνται σε partitions βάσει αυτού του κλειδιού κατακερματισμού. Ορίζουμε ως $Part(s)$ τον αριθμό των partitions μίας διαδικασίας shuffling s. Αυτή η διαδικασία θα ολοκληρωθεί σε αριθμό διεργασιών που ισούται με $Part(s)$ και κάθε διεργασία θα λάβει ως είσοδο $\frac{R(s)}{Part(s)}$ γραμμές. Το κόστος αυτής της διαδικασίας ορίζεται από την παρακάτω εξίσωση:

$$C_{shuffle}(s) = \sum_{n=1}^{Rounds(Part(s))} \max_n \left(\frac{R(s)}{Part(s)} \cdot (C_{cpu} + D_w) \right)$$

Κόστος Broadcast Hash Join

Πρόκειται για έναν αποδοτικό αλγόριθμο συνένωσης στον οποίο ο μικρότερος από τους δύο πίνακες κατακερματίζεται στο κλειδί που χρησιμοποιείται στην συνθήκη συνένωσης. Έπειτα ο κατακερματισμένος πίνακας μεταδίδεται σε όλους τους workers στους οποίους συνενώνεται με μικρότερα τμήματα του μεγαλύτερου πίνακα. Ο συγκεκριμένος αλγόριθμος σε ορισμένες περιπτώσεις μπορεί να πετύχει σημαντική αύξηση της απόδοσης συγκριτικά με τον Sort-Merge Join καθώς παρακάμπτει την διαδικασία της ταξινόμησης. Το κόστος αναλύεται στα εξής μέρη:

1. **Hash & Broadcast**: Αυτή η διαδικασία λαμβάνει χώρο στον driver node του Spark. Όλες οι γραμμές κατακερματίζονται σειριακά με κόστος t_h. Έπειτα κάθε γραμμή μεταδίδεται με κόστος t_h.

$$C_{broadcast}(s) = R(s) \cdot (t_h + t_{br}) \tag{Ε΄.3}$$

2. **Join**: Σε αυτό το βήμα, αφού ο μικρότερος πίνακας έχει μεταδοθεί σε όλους τους κόμβους συνενώνεται με κάθε τμήμα του μεγαλύτερου. Το κόστος ορίζεται από την παρακάτω εξίσωση:

$$C_{bhj} = C_{broadcast}(s) + \sum_{n=1}^{Rounds(Part(l))} \max_n \left(\frac{R(s) \cdot R(l) \cdot C_{cpu}}{Part(l)} \right) \tag{Ε΄.4}$$

Κόστος Sort-Merge Join

Σε αυτόν τον αλγόριθμο, πριν την συνένωση, στον κάθε πίνακα εφαρμόζεται μία διαδικασία shuffling για τον σωστό καταμερισμό των γραμμών και μετά κάθε πίνακας ταξινομείται. Το κόστος αναλύεται στα 3 εξής τμήματα:

1. **Shuffle**: Το κόστος αυτής της διαδικασίας είναι όμοιο με αυτό της εξίσωσης Ε΄.0.2.

2. **Sort**: Στην πορεία οι δύο πίνακες ταξινομούνται. Το κόστος για κάθε πίνακα ισούται με

$$C_{sort}(s) = \sum_{n=1}^{Rounds(Part(s))} \max_{n} \left(R(s) \cdot \log R(s) \cdot C_{cpu}\right) \tag{E΄.5}$$

3. **Merge**: Έπειτα, μετά από το shuffling και την ταξινόμηση οι δύο πίνακες συγχωνεύονται με κόστος:

$$C_{merge}(s,t) = \sum_{n=1}^{n} \max_{n} \left(\frac{R(s)}{Part(s)} \cdot \frac{R(t)}{Part(t)} \cdot C_{cpu}\right) \tag{E΄.6}$$

4. Συνοψίζοντας, το συνολικό κόστος του αλγορίθμου περιγράφεται από την παρακάτω εξίσωση

$$C_{smj}(s,t) = C_{shuffle}(s) + C_{sort}(s) + C_{shuffle}(t) + C_{sort}(t) + C_{merge}(s,t) \tag{E΄.7}$$

Κεφάλαιο ΣΤ΄

Πειράματα

Σε αυτό το κεφάλαιο παρουσιάζονται τα πειράματα τα οποία εκτελέστηκαν προκειμένου να γίνει η αξιολόγηση της απόδοσης του συστήματος που αναπτύχθηκε. Αρχικά περιγράφονται συνοπτικά τα συστήματα στα οποία έτρεξε το MuSQLE, δηλαδή την MemSQL, την PostgreSQL καθώς και το Hadoop Distributed File System (HDFS). Στην πορεία την απόδοση του MuSQLE με αυτά τα συστήματα. Στην πορεία παρουσιάζονται τα αποτελέσματα των πειραμάτων που αξιολογούν την απόδοση χρόνου βελτιστοποίησης, την ακρίβεια των μοντέλων κόστους καθώς και την απόδοση της εκτέλεσης των ερωτημάτων.

Μηχανές

MemSQL

Η MemSQL είναι μία σχεσιακή βάση δεδομένων η οποία λειτουργεί εξ'ολοκλήρου στην κύρια μνήμη. Αντίθετα με άλλες σχεσιακές βάσεις δεδομένων όπως η PostgreSQL ή η MonetDB που τα δεδομένα βρίσκονται στον δίσκο στην MemSQL είναι διαθέσιμα πάντα στην κύρια μνήμη. Αυτό έχει ως αποτέλεσμα την ταχύτερη εκτέλεση ενός ερωτήματος καθώς ελαχιστοποιούνται τα I/Os στον δίσκο. Επιπλέον χαρακτηριστικό που συμβάλει στην μεγάλη ταχύτητα εκτέλεσης που προσφέρει είναι πως μετατρέπει απευθείας τα ερωτήματα από SQL κώδικα σε κώδικα μηχανής.

PostgreSQL

Η PostgreSQL είναι μία ευρέως γνωστή, ανοιχτού κώδικα, σχεσιακή βάση δεδομένων (RDBMS) με έμφαση στην επεκτασιμότητα καθώς και στην αυστηρή εφαρμογή των προτύπων από τα οποία ορίζεται ένα RDBMS. Καθώς πρόκειται για database server μία από τις κύριες λειτουργίες της είναι η ασφαλής αποθήκευση των δεδομένων καθώς και η ανάκτηση τους από αιτήματα εξωτερικών εφαρμογών. Μπορεί να διαχειριστεί από μικρούς φόρτους εργασίας σε μία μηχανή έως και μεγάλες διαδικτυακές εφαρμογές ως αποθήκη δεδομένων υποστηρίζοντας την ταυτόχρονη χρήση της από πολλούς χρήστες παράλληλα. Επιπλέον η PostgreSQL είναι μία transactional βάση δεδομένων τηρώντας το μοντέλο ACID(Atomicity, Consistency, Isolation, Durability).

Hadoop Distributed File System (HDFS)

Το Hadoop Distributed File System είναι ένα από τα τέσσερα τμήματα που απαρτίζουν το Apache Hadoop Project. Πρόκειται για ένα κατανεμημένο σύστημα αρχείων στο οποίο τα δεδομένα τεμαχίζονται σε μικρότερα τμήματα(blocks) και κατανέμονται στο cluster. Αυτό δίνει την δυνατότητα για γρήγορη κατανεμημένη επεξεργασία σε δεδομένα μεγάλης κλίμακας με το προγραμματιστικό μοντέλο MapReduce.

Data Replication

Τα τμήματα που προκύπτουν από την διαίρεση των δεδομένων μπορεί διατίθενται σε παραπάνω από ένα αντίγραφα, συγκεκριμένα σε τόσα όσα ορίζει η παράμετρος *replication factor*. Αυτό προσφέρει τα εξής οφέλη:

- Ανοχή σε σφάλματα: Αντίγραφο του κάθε τμήματος δεδομένων μπορεί να βρίσκεται σε παραπάνω από έναν server (3 εξ ορισμού). Έτσι, στην περίπτωση που κάποιος κόμβος υποστεί βλάβη υπάρχουν τα τμήματα που χάθηκαν διαθέσιμα σε άλλους 2. Σε αυτή την περίπτωση το σύστημα δημιουργεί ένα τρίτο αντίγραφο σε κάποιον άλλο υγιή κόμβο ώστε να ικανοποιείται το replication factor.

- Υψηλή διαθεσιμότητα: Διατηρώντας τμήματα των αρχείων σε πολλούς κόμβους το κάθε τμήμα είναι ταυτόχρονα διαθέσιμο σε αρκετά αντίγραφα στην περίπτωση που κάποιος κόμβος που διαθέτει κάποιο τμήμα έχει υψηλό φόρτο εργασίας.

Αρχιτεκτονική

Το HDFS ακολουθεί την αρχιτεκτονική master/slave, που σημαίνει πως υπάρχει ένας κόμβος ο οποίος λειτουργεί ως master node και πλήθος από slaves ως DataNodes, οι οποίοι κρατούν τα δεδομένα και διαχειρίζονται από τον master. Παρακάτω αναλύονται τα επιμέρους τμήματα της αρχιτεκτονικής:

- Namenode: Ο Namenode είναι το κεντρικό τμήμα όλου το HDFS. Είναι το σημείο με το οποίο επικοινωνούν όλες οι εφαρμογές πελάτη προκειμένου να εκτελέσουν λειτουργίες στο σύστημα αρχείων. Στην περίπτωση που υπάρξει βλάβη στον Namenode όλο το HDFS παύει να είναι διαθέσιμο. Συγκεκριμένα, είναι ένας κόμβος στο δίκτυο ο οποίος διαχειρίζεται όλους τους υπόλοιπους slave κόμβους στο HDFS cluster. Κατά την εκκίνησή του διαβάζει το σύστημα αρχείων διαβάζοντας το *FsImage* το οποίο διατηρεί metadata τα οποία περιγράφουν τα ίδια τα δεδομένα και περιέχουν πληροφορίες σχετικές με τα blocks, δηλαδή πόσα υπάρχουν (replication) και που είναι διαθέσιμο το καθένα από αυτά. Επιπλέον, μια σημαντική λειτουργία του namenode είναι ο έλεγχος της κατάστασης των slaves, δηλαδή ποιοι είναι υγιείς, ποιοι όχι, πότε στάλθηκε το τελευταίο heartbeat από τον καθένα.

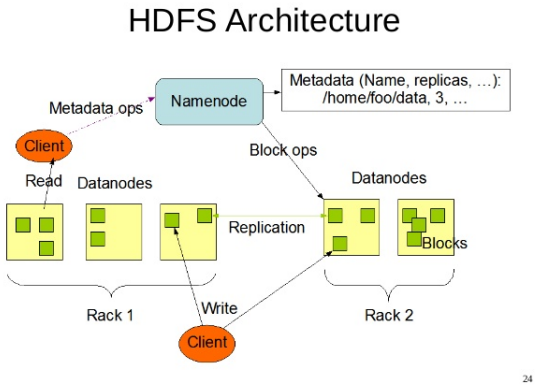
- Datanodes: Οι datanodes διατηρούν τα blocks των δεδομένων και εκτελούν λειτουργίες/αιτήματα εγγραφής και ανάγνωσης. Στην εκκίνηση τους συνδέονται με τον Namenode και ξεκινάει το service στον κόμβο να τρέχει. Επιπλέον, οι datanodes έπειτα από εντολή από τον Namenode διεκπεραιώνουν λειτουργίες όπως δημιουργία, αντιγραφή και διαγραφή των blocks.

Στο σχήμα ΣΤ΄.7 φαίνεται αναλυτικά η αρχιτεκτονική του HDFS.

Υποδομή

Η υποδομή στην οποία εκτελέστηκαν τα πειράματα αποτελείται από τρεις διαφορετικού τύπου μηχανές: Ένα Spark cluster, ένα MemSQL cluster, μία PostgreSQL καθώς και ένα HDFS cluster για την αποθήκευση των Parquet δεδομένων. Πιο αναλυτικά για την κάθε μηχανή ισχύουν τα εξής:

- Spark SQL
  - 1 Master node, 4VCPUs, 8GB RAM



Σχήμα ΣΤ΄.7: *Η αρχιτεκτονική του HDFS*

  - 5 Worker nodes, 4VCPUs, 8GB RAM/node

- HDFS
  - 1 Namenode, 4VCPUs, 8GB RAM
  - 5 Datanodes, 4VCPUs, 8GB RAM/node
- MemSQL
  - 1 Master aggregator, 4VCPUs, 8GB RAM
  - 5 Child aggregators, 4VCPUs, 8GB RAM/node
- PostgreSQL
  - 1 Virtual Machine, 4VCPUs, 8GB RAM

Σενάρια

Προκειμένου να δοκιμαστεί η απόδοση του συστήματος σε διάφορες συνθήκες σχεδιάστηκαν τα εξής σενάρια που στοχεύουν σε διαφορετικά χαρακτηριστικά του συστήματος:

- Στο πρώτο σενάριο όλοι οι πίνακες είναι διαθέσιμοι σε όλες τις μηχανές. Αυτό το σενάριο στοχεύει στην ακρίβεια του μοντέλου κόστους της κάθε μηχανής, καθώς στις πε-

Table	Location
customer	PostgreSQL
nation	PostgreSQL
region	PostgreSQL
part	MemSQL
partsupp	MemSQL
orders	HDFS
lineitem	HDFS

Πίνακας ΣΤ΄.1: *Τοποθεσίες των πινάκων για το δεύτερο σενάριο πειραμάτων*

ρισσότερες περιπτώσεις δεν πραγματοποιούνται μετακινήσεις των ενδιάμεσων αποτελεσμάτων, αλλά εκτελείται όλο το ερώτημα σε μία μηχανή. Κατά συνέπεια, σε αυτά τα πειράματα εξετάζεται το αν το σύστημα σε κάθε περίπτωση έκανε την βέλτιστη επιλογή.

- Στο δεύτερο σενάριο κάθε πίνακας είναι διαθέσιμος σε μία μηχανή. Σε αυτά τα πειράματα εξετάζεται αν το αρχικό ερώτημα χωρίστηκε στα κατάλληλα σημεία του δέντρου και αν επιλέχθηκε η βέλτιστη μηχανή για την εκτέλεση του κάθε υποερωτήματος. Οι τοποθεσίες των πινάκων για αυτό το σενάριο απεικονίζονται στον πίνακα ΣΤ΄.1

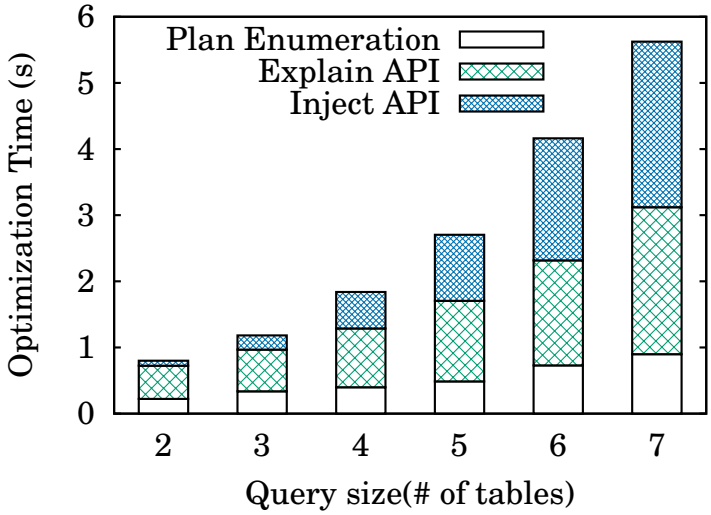
Ως δεδομένα εισόδου χρησιμοποιήθηκαν πίνακες που δημιουργήθηκαν με το γνωστό TPC-H benchmark [6]. Δημιουργήθηκαν δεδομένα τριών διαφορετικών μεγεθών των 5, 20 και 50 Gigabytes.

Για τα πειράματα έτρεξαν 17 ερωτήματα [1] τα οποία γράφτηκαν για τις ανάγκες της εργασίας. Τα ερωτήματα χωρίζονται σε 2 κατηγορίες:
Join-Only(Q0 - Q8): Ερωτήματα που αποτελούνται μόνο από joins στο κύριο κλειδί
Join-Filter(Q9 - Q17): Ερωτήματα που αποτελούνται από joins στο κύριο κλειδί καθώς και ορισμένα φίλτρα που έχουν ως αποτέλεσμα μικρό αριθμό γραμμών.

Απόδοση του Planner

Σε αυτά τα πειράματα παρουσιάζεται η απόδοση του planner για τα εξής σενάρια:

[1] https://github.com/gsvic/MuSQLE/blob/master/Queries.scala



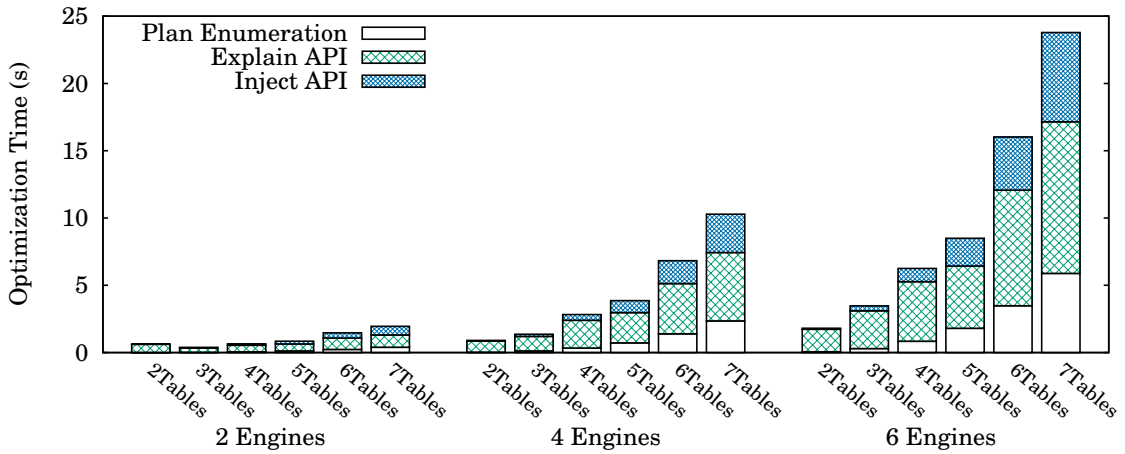
Σχήμα ΣΤ΄.8: *Απόδοση του planner για 3 μηχανές*

3 Μηχανές

Στο διάγραμμα ΣΤ΄.8 απεικονίζονται οι χρόνοι εκτέλεσης του planner για τον σχεδιασμό του βέλτιστου πλάνου σε ένα περιβάλλον που αποτελείται από 3 μηχανές. Στην καλύτερη περίπτωση για ένα ερώτημα που αναφέρεται σε 2 πίνακες ο χρόνος εκτέλεσης είναι σχεδόν ένα δευτερόλεπτο, ενώ στην χειρότερη περίπτωση για 7 πίνακες ο χρόνος φτάνει σχεδόν τα 6 δευτερόλεπτα. Αυτό συμβαίνει κυρίως λόγω του μεγάλου αριθμού των κλήσεων στο Engine API στις μεθόδους *getCost* και *injectStats* στην κάθε μηχανή. Αυτό μπορεί να φανεί καθαρά στα διαγράμματα καθώς σε κάθε μπάρα χωρίζονται οι χρόνοι εκτέλεσης από κάθε ξεχωριστή λειτουργία της διαδικασίας του planning. Ο χρόνος εκτέλεσης του αλγορίθμου DPhyp απομονωμένος, χωρίς τις άλλες λειτουργίες ξεκινά από τα 0.22 δευτερόλεπτα και φτάνει έως τα 0.89.

6 Μηχανές

Έπειτα από το πείραμα με τις 3 μηχανές θα θέλαμε να δείξουμε την απόδοση του planner σε σενάρια με περισσότερες από αυτές στην περίπτωση που το σύστημα τρέχει σε κάποια πιο πολύπλοκη υποδομή. Για να το πετύχουμε αυτό έγιναν τα εξής: Σχεδιάστηκαν 6 πλασματικές μηχανές που υλοποιούν το Engine API. Έπειτα, για όλες τις προηγούμενες πραγματικές μηχανές μετρήθηκε ο μέσος όρος του χρόνου που απαιτείται για μία κλήση της συνάρτησης getCost, καθώς και της συνάρτησης injectStats. Έπειτα, αυτός ο χρόνος καθυστέρησης προσομοιώνεται

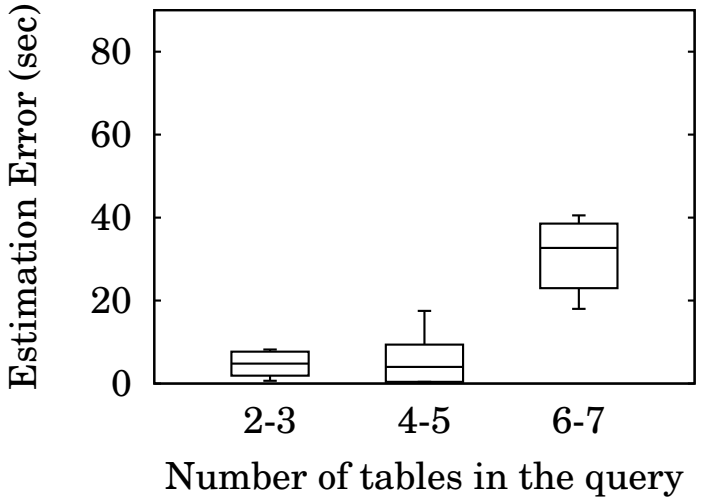


Σχήμα ΣΤ΄.9: *Απόδοση του planner για 6 μηχανές*

στις πλασματικές μηχανές με την χρήση της συνάρτησης *sleep*. Στην εικόνα ΣΤ΄.9 απεικονίζονται τα αποτελέσματα αυτού του πειράματος. Στους χρόνους εκτέλεσης μπορούμε να δούμε πως για 2 πίνακες ο χρόνος του planning είναι λίγο παραπάνω από 1 δευτερόλεπτο ενώ στην χειρότερη περίπτωση των 6 πινάκων ο χρόνος αυξάνεται σημαντικά λόγω των περισσότερων μηχανών και αγγίζει σχεδόν τα 25 δευτερόλεπτα. Και σε αυτό το πείραμα διακρίνουμε πως τον περισσότερο χρόνο του planning κατέχουν οι πολλαπλές κλήσεις των μεθόδων *getCost* και *injectStats*. Η πραγματική απαρίθμηση των πλάνων εκτέλεσης από τον DPhyp ξεκινάει απο τα 0.06 δευτερόλεπτα και φτάνει έως και τα 5.88.

Ακρίβεια εκτίμησης κόστους

Σε αυτή την ενότητα παρουσιάζονται τα πειράματα για την ακρίβεια του μοντέλου κόστους τριών μηχανών που χρησιμοποιούνται στην παρούσα εργασία. Όπως αναφέρθηκε και σε προηγούμενο κεφάλαιο, αυτές οι μηχανές επιστρέφουν το κόστος εκτέλεσης σε διαφορετικές μονάδες μέτρησης. Προκειμένου να μπορούμε να συγκρίνουμε τα διάφορα κόστη μεταξύ τους χρησιμοποιήσαμε μοντέλα παλινδρόμησης προκειμένου να μετατρέψουμε κάθε μονάδα μέτρησης του κόστους σε χρόνο. Στα box plots παρακάτω παρουσιάζεται η ακρίβεια του μοντέλου της MemSQL (εικόνα ΣΤ΄.11), της PostgreSQL (εικόνα ΣΤ΄.12) καθώς και του μοντέλου κόστους που αναπτύξαμε για το Spark SQL (εικόνα ΣΤ΄.10). Στα box plots απεικονίζονται



Σχήμα ΣΤ΄.10: *Ακρίβεια μοντέλου κόστους του Spark SQL*

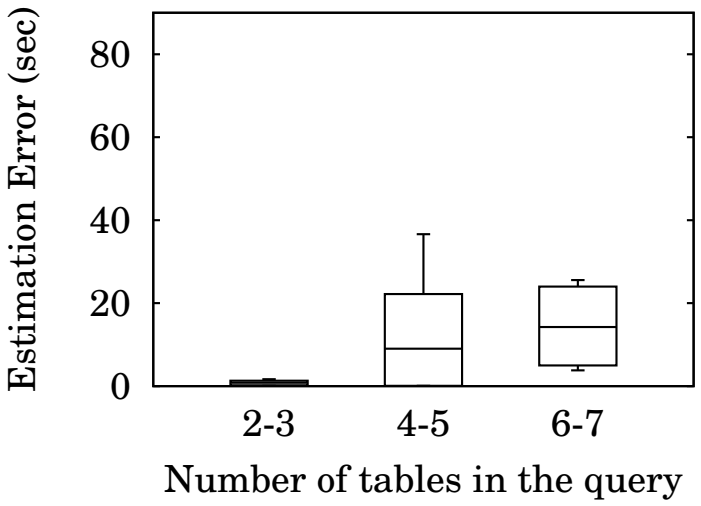
για το σφάλμα εκτίμησης της κάθε μηχανής ο μέσος όρος, η τυπική απόκλιση καθώς και οι ελάχιστη και η μέγιστη τιμή. Έχουμε ομαδοποιήσει την πολυπλοκότητα(αριθμός πινάκων που εμπλέκονται στο ερώτημα) του κάθε πειράματος σε 3 ομάδες: 2-3, 4-5, και 6-7 πίνακες. Σε όλα τα πειράματα, όπως είναι αναμενόμενο, το σφάλμα εκτίμησης αυξάνεται όσο αυξάνονται οι πίνακες που εμπλέκονται στο ερώτημα. Αυτό συμβαίνει διότι όσο αυξάνεται η πολυπλοκότητα του πειράματος αυξάνεται και το σφάλμα εκτίμησης της πληθικότητας(cardinality). Ωστόσο, συγκριτικά με τις άλλες δύο μηχανές το μοντέλο κόστους που προτείνεται για το Spark SQL πετυχαίνει ακριβείς εκτιμήσεις.

Απόδοση εκτέλεσης ερωτήματος

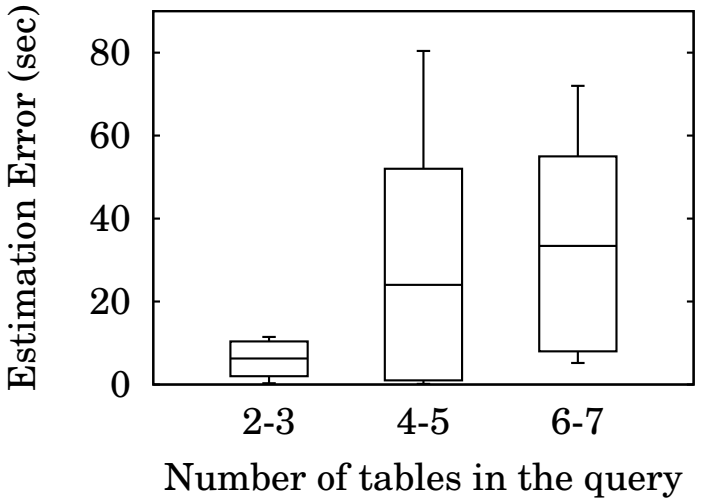
Σε αυτή την ενότητα παρουσιάζονται τα αποτελέσματα των πειραμάτων για την απόδοση του χρόνου εκτέλεσης των ερωτημάτων στο MuSQLE. Το σύστημα συγκρίνεται με 3 μηχανές: Την PostgreSQL, την MemSQL και το Spark SQL.

Όλοι οι πίνακες σε όλες τις μηχανές - 5GB

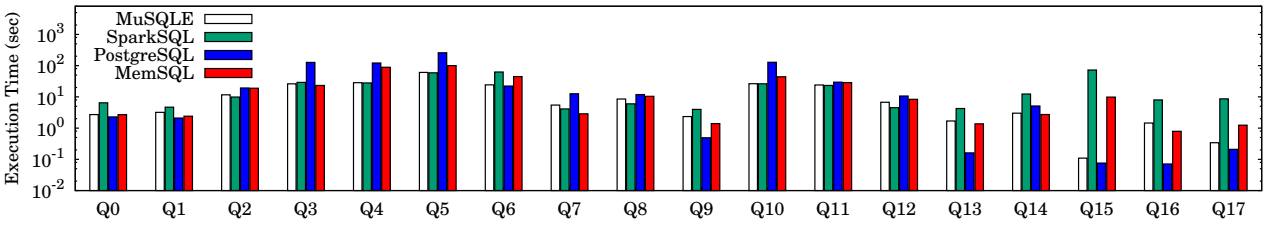
Στην εικόνα ΣΤ΄.13 απεικονίζονται οι χρόνοι εκτέλεσης για τα πειράματα στα οποία όλοι οι πίνακες είναι διαθέσιμοι σε όλες τις μηχανές, για το TPC-H dataset των 5GB. Σε αυτό το πεί-



Σχήμα ΣΤ΄.11: *Ακρίβεια μοντέλου κόστους της MemSQL*



Σχήμα ΣΤ΄.12: *Ακρίβεια μοντέλου κόστους της PostgreSQL*



Σχήμα ΣΤ΄.13: *Όλοι οι πίνακες σε όλες τις μηχανές - 5GB*

ραμα φαίνεται πως σε όλα τα ερωτήματα το MuSQLE δεν πετυχαίνει καλύτερη απόδοση από την μηχανή η οποία είχε τον καλύτερο χρόνο εκτέλεσης. Αυτό συμβαίνει διότι λόγω του μικρού μεγέθους των δεδομένων καθώς και την διαθεσιμότητας των πινάκων σε όλες τις μηχανές, σε όλες τις περιπτώσεις είναι προτιμότερο το ερώτημα να εκτελεστεί εξ'ολοκλήρου σε μία μηχανή και να μην υπάρξουν μετακινήσεις των ενδιάμεσων αποτελεσμάτων μεταξύ των μηχανών. Ωστόσο, από τα διαγράμματα φαίνεται πως το MuSQLE επιλέγει βάσει του μοντέλου κόστους την μηχανή που πετυχαίνει τον βέλτιστο χρόνο εκτέλεσης. Στα διαγράμματα φαίνεται πως ο χρόνος εκτέλεσης του MuSQLE στις περισσότερες περιπτώσεις είναι κοντά στον χρόνο εκτέλεσης της μηχανής με την υψηλότερη απόδοση, με ένα επιπλέον κόστος λόγω του planning που σε πολλές περιπτώσεις καθυστερεί αρκετά λόγω της πολυπλοκότητας των ερωτημάτων και τον πολλαπλών κλήσεων των μεθόδων του Engine API. Σε αυτό το πείραμα φαίνεται κυρίως η ακρίβεια του planner στην λήψη αποφάσεων.

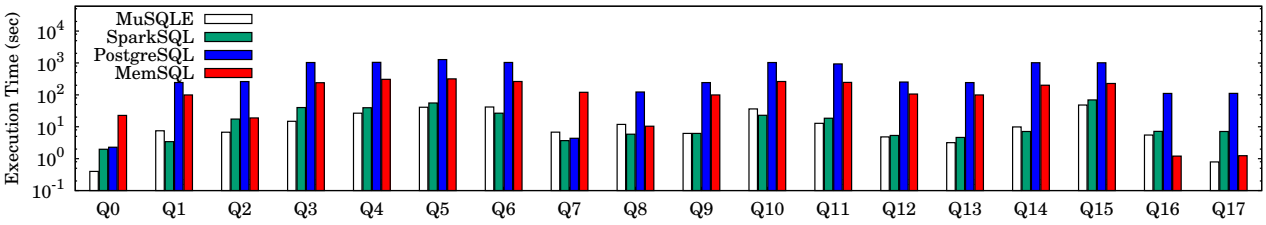
Στην συνέχεια αναλύονται οι χρόνοι εκτέλεσης για τα πειράματα στα οποία οι πίνακες είναι κατανεμημένοι στις μηχανές με τις τοποθεσίες που αναγράφονται στον πίνακα ΣΤ΄.1.

Κατανεμημένοι πίνακες - 5GB

Στην εικόνα ΣΤ΄.14 παρουσιάζονται οι χρόνοι εκτέλεσης για τα δεδομένα μεγέθους 5GB. Σε αυτό το πείραμα, λόγω του μικρού μεγέθους των δεδομένων, στις περισσότερες περιπτώσεις το βέλτιστο πλάνο είναι να μετακινηθούν τα δεδομένα σε μία μηχανή και να εκτελεστεί το ερώτημα εκεί. Ωστόσο, και σε αυτό το πείραμα φαίνεται κυρίως η ακρίβεια του συστήματος στην σωστή λήψη αποφάσεων.

Κατανεμημένοι πίνακες - 20,50GB

Σε αυτά τα πειράματα τα δεδομένα εισόδου είναι αρκετά μεγαλύτερα, μεγέθους 20 και 50 GB. Καθώς το μέγεθος αυξάνεται, η απόδοση των τριών μηχανών που συγκρίνουμε το σύστημά



Σχήμα ΣΤ΄.14: *Κάθε πίνακας σε μία μηχανή - 5GB*

μας πέφτει σημαντικά. Αυτό συμβαίνει διότι πριν την εκτέλεση του ερωτήματος κάθε μηχανή χρειάζεται να φέρει μέσω δικτύου τους πίνακες που αναφέρονται στο ερώτημα αλλά δεν διαθέτει. Όταν ο πίνακας είναι μεγάλος αυτός ο χρόνος αυξάνεται σημαντικά και κατά συνέπεια και ο συνολικός χρόνος εκτέλεσης του ερωτήματος.

PostgreSQL: Υπάρχουν περιπτώσεις στις οποίες η PostgreSQL χρειάστηκε πάνω από 20 λεπτά για την εκτέλεση του ερωτήματος, καθώς η διαδικασία φόρτωσης δεδομένων έχει μεγάλο κόστος εκτέλεσης λόγω της μεταφοράς μέσω δικτύου αλλά και λόγω της εγγραφής των δεδομένων στον δίσκο. Αυτές οι περιπτώσεις σημειώνονται στα ιστογράμματα στις εικόνες ΣΤ΄.15 και ΣΤ΄.16 με ένα **x**.

MemSQL: Η MemSQL, καθώς πρόκειται για βάση δεδομένων κύριας μνήμης απαιτεί πολύ λιγότερο χρόνο για την μεταφορά των δεδομένων. Συγκεκριμένα ο χρόνος που απαιτείται είναι κυρίως αυτός της μεταφοράς μέσω δικτύου, καθώς τα δεδομένα δεν χρειάζεται να εγγραφούν στον δίσκο αλλά μένουν μονίμως στην κύρια μνήμη. Ωστόσο, λόγω του περιορισμένου χώρου της κύριας μνήμης, για δεδομένα μεγάλου μεγέθους υπάρχουν σφάλματα κατά την μεταφορά τους καθώς αυτά δεν χωράνε στην μνήμη. Επιπλέον, σε πολλές περιπτώσεις το σύστημα αποτυγχάνει στην ολοκλήρωση του ερωτήματος, καθώς υπάρχει ελάχιστος χώρος για τα ενδιάμεσα αποτελέσματα που παράγονται αφού ήδη κατέχουν τον περισσότερο χώρο οι αρχικοί πίνακες. Ως αποτέλεσμα, σε τέτοιες περιπτώσεις οι κόμβοι μπαίνουν σε κατάσταση εκτός λειτουργίας λόγω σφαλμάτων της κύριας μνήμης. Αυτές οι περιπτώσεις είναι σημειωμένες στα διαγράμματα ΣΤ΄.15 και ΣΤ΄.16 με **o(out-of-memory)**

Spark SQL: Το Spark SQL λόγω του κατανεμημένου μοντέλου επεξεργασίας που διαθέτει σε όλες τις περιπτώσεις εκτελεί επιτυχώς τα ερωτήματα μέχρι τέλους. Το Spark SQL επεξεργάζεται τα ερωτήματα στην κύρια μνήμη, και στην περιπτώσεις που τμήμα αυτών δεν χωράει σε αυτή γράφεται στον σκληρό δίσκο, και ανακτάται στην πορεία όταν αυτό χρειαστεί. Ωστόσο το Spark SQL σε όλες τις περιπτώσεις χρειάζεται να φέρει όλα τα δεδομένα από τις εξωτερικές μηχανές μέσω δικτύου, πράγμα το οποίο σε ορισμένες περιπτώσεις επηρεάζει σημαντικά

την απόδοση. Στην περίπτωση της PostgreSQL για παράδειγμα πρέπει πρώτα τα δεδομένα να φορτωθούν σε τμήματα στην κύρια μνήμη από τον δίσκο και μετά να σταλούν μέσω του δικτύου. Στην MemSQL τα δεδομένα είναι πάντα στην κύρια μνήμη, οπότε εκεί η καθυστέρηση οφείλεται μόνο στην μεταφορά και επηρεάζεται καθαρά από το bandwidth.

MuSQLE: Αντίθετα με το Spark SQL, το MuSQLE ακολουθεί την πρακτική που παρουσιάζεται στην υποενότητα Δ΄.4. Συγκεκριμένα, αυτό που κάνει είναι τα σπάει τον γράφο του ερωτήματος σε όλους τους δυνατούς υπογράφους και με τη χρήση των μοντέλων κόστους επιλέγει που θα εκτελέσει το κάθε υποερώτημα. Έπειτα, για την εκτέλεση του τελικού ερωτήματος γίνονται μεταφορές των ενδιάμεσων αποτελεσμάτων στην μηχανή η οποία πρόκειται να το εκτελέσει. Για παράδειγμα, στο ερώτημα Q_8 το οποίο ανήκει στην κατηγορία των *join-only* ερωτημάτων:

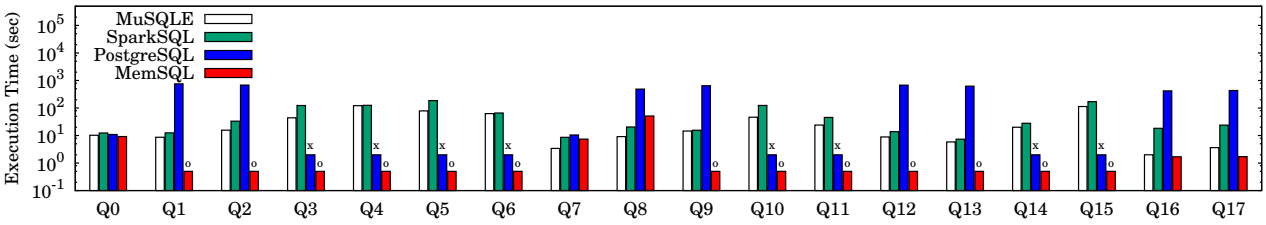
```
SELECT p_name
FROM part, partsupp
WHERE p_partkey = ps_partkey
```

Το Spark SQL θα μετέφερε τα δεδομένα στο cluster του και έπειτα θα εκτελούσε το ερώτημα στο δικό του σύστημα με την χρήση των RDDs. Αντιθέτως, το MuSQLE θα στείλει τον παραπάνω SQL κώδικα για εκτέλεση στην MemSQL και θα λάβει τον πίνακα με το αποτέλεσμα, εκμεταλλευόμενο τις δυνατότητες της εκτέλεσης στην κύρια μνήμη της MemSQL πετυχαίνοντας σε μεγάλο βαθμό καλύτερη απόδοση από το Spark SQL. Ένα άλλο παράδειγμα είναι το Q_{13}, που πρόκειται για ένα *join-filter* ερώτημα:

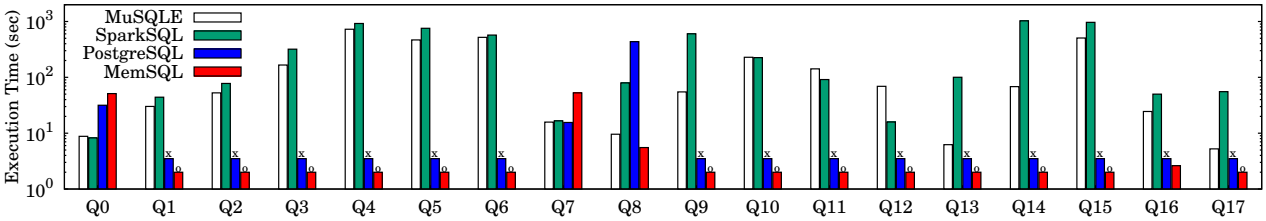
```
SELECT o_orderkey
FROM orders, customer, nation, region
WHERE o_custkey = c_custkey
AND c_nationkey = n_nationkey
AND n_regionkey = r_regionkey
AND o_orderkey < 100
```

Σε αυτή την περίπτωση ο ένας υπογράφος του ερωτήματος αναπαρίσταται από το εξής υποερώτημα:

```
SELECT o_orderkey
FROM orders
AND o_orderkey < 100
```



Σχήμα ΣΤ΄.15: *Κάθε πίνακας σε μία μηχανή - 20GB*



Σχήμα ΣΤ΄.16: *Κάθε πίνακας σε μία μηχανή - 50GB*

το οποίο αναφέρεται στο scan και στο φίλτρο του πίνακα orders. Σε αυτή την περίπτωση το MuSQLE εκτελεί στο Spark SQL το συγκεκριμένο υποερώτημα το οποίο έχει αποτέλεσμα 99 γραμμές. Έπειτα το μεταφέρει στην PostgreSQL για να εκτελέσει εκεί το join με τους υπόλοιπους τρεις πίνακες, πετυχαίνοντας και πάλι σημαντικά καλύτερη απόδοση από το Spark SQL.

Κεφάλαιο Ζ΄

Συμπεράσματα

Σε αυτή την εργασία παρουσιάστηκε το σύστημα MuSQLE, ένα σύστημα με στόχο την βελτιστοποίηση επερωτήσεων σε περιβάλλοντα πολλαπλών μηχανών. Το MuSQLE είναι βασισμένο στο Spark SQL, προσφέροντας την δυνατότητα της σύνθεσης ερωτημάτων που αναφέρονται σε πίνακες σε περισσότερες από μία διαφορετικές μηχανές. Επιπλέον το MuSQLE με την χρήση του Engine API δίνει την δυνατότητα στον χρήστη να προσθέσει μία νέα μηχανή στο σύστημα υλοποιώντας τις 4 βασικές μεθόδους που το περιγράφουν. Επιπλέον, με τη χρήση του βελτιστοποιητή πολλαπλών μηχανών το MuSQLE βελτιστοποιεί τα ερωτήματα λαμβάνοντας υπόψιν το μοντέλο κόστους της κάθε μηχανής. Όπου είναι δυνατό, το MuSQLE στέλνει τα επιμέρους υποερωτήματα για τοπική εκτέλεση στις μηχανές στις οποίες τρέχει. Έπειτα, πραγματοποιεί μετακινήσεις των ενδιάμεσων αποτελεσμάτων μεταξύ των μηχανών για την ολοκλήρωση της εκτέλεσης του ερωτήματος. Από την πειραματική αξιολόγηση φαίνεται πως το MuSQLE μπορεί να αποφασίσει με ακρίβεια που θα εκτελεστεί το κάθε υποερώτημα αξιοποιώντας όλες τις μηχανές στις οποίες τρέχει, πετυχαίνοντας με αυτόν τον τρόπο βελτίωση απόδοσης έως και μία τάξη μεγέθους.

Κεφάλαιο Η΄

Δημοσιεύσεις

- **Giannakouris, V.**, Papailiou, N., Tsoumakos, D., Koziris, N. **"MuSQLE: Distributed SQL Query Execution Over Multiple-Engine Environments".** *In proceedings of the 2016 IEEE International Conference on Big Data (BigData 2016), Washington, DC, USA, December 5-8 2016*

- Doka, K., Papailiou, N., **Giannakouris, V.**, Tsoumakos, D., Koziris, N. **"Mix 'n Match Multi-Engine Analytics".** *In proceedings of the 2016 IEEE International Conference on Big Data (BigData 2016), Washington, DC, USA, December 5-8 2016*

- Papailiou, N., Doka, K., **Giannakouris, V.**, Papaioannou, V., Tsoumakos, D., Koziris, N. **"Optimizing, Planning and Executing Analytics Workflows over Multiple Engines."** *In Proceedings of the 10th International Workshop on Enabling Real-Time Business Intelligence (BIRTE 2016), in conjunction with the VLDB 2016 Conference, September 9, 2016, New Delhi, India.*

- Doka K., Filatov M., **Giannakouris, V.**, Kantere, V., Koziris, N., Mantas, C., Papailiou, N., Papaioannou, P., Tsoumakos, D. **"Optimizing, Planning and Executing Analytics Workflows over Multiple Engines."** *1st International Workshop on Multi-Engine Data AnaLytics (MEDAL 2016), Bordeaux, France, 2016*

Βιβλιογραφία

[1] Amazon S3. https://en.wikipedia.org/wiki/Amazon$_S$3.

[2] Instagram. https://www.instagram.com.

[3] PostgreSQL. https://www.postgresql.org/.

[4] Redis. https://redis.io/.

[5] Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., et al. Spark sql: Relational data processing in spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (2015), ACM, pp. 1383–1394.

[6] Council, T. P. P. Tpc-h benchmark specification. *Published at http://www. tcp. org/hspec. html* (2008).

[7] Duggan, J., Elmore, A. J., Stonebraker, M., Balazinska, M., Howe, B., Kepner, J., Madden, S., Maier, D., Mattson, T., and Zdonik, S. The bigdawg polystore system. *ACM Sigmod Record 44*, 2 (2015), 11–16.

[8] Fender, P., and Moerkotte, G. Top down plan generation: From theory to practice. In *Data Engineering (ICDE), 2013 IEEE 29th International Conference on* (2013), IEEE, pp. 1105–1116.

[9] Giannakouris, V., Papailiou, N., Tsoumakos, D., and Koziris, N. Musqle: Distributed sql query execution over multiple engine environments. In *IEEE Big Data (Big Data)* (2016).

[10] Kolev, B., Valduriez, P., Bondiombouy, C., Jiménez-Peris, R., Pau, R., and Pereira, J. Cloudmdsql: Querying heterogeneous cloud data stores with a common language. *Distributed and Parallel Databases* (2015), 1–41.

[11] LeFevre, J., Sankaranarayanan, J., Hacigumus, H., Tatemura, J., Polyzotis, N., and Carey, M. J. Miso: souping up big data query processing with a multistore system. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data* (2014), ACM, pp. 1591–1602.

[12] Moerkotte, G., and Neumann, T. Dynamic programming strikes back. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data* (2008), ACM, pp. 539–552.

[13] Taniar, D., Leung, C. H., Rahayu, W., and Goel, S. *High performance parallel database processing and grid databases*, vol. 67. John Wiley & Sons, 2008.

[14] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation* (2012), USENIX Association, pp. 2–2.

[15] Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., and Stoica, I. Spark: cluster computing with working sets. *HotCloud 10* (2010), 10–10.